Generative Models

From First Principles to Frontier Research

Yogesh P

February 2026

Overview

All chapters at a glance

Ch. 0 Introduction

Three ways to think about generating images: the pixel view, the feature view, and the noise view.

Part I: Explicit Likelihood Models

Ch. 1 Autoregressive Models

From FVSBN to Transformers: fixed-length models over images and audio, and variable-length sequence models.

Contents

— Part I: Explicit Likelihood Models — **4**

Chapter 1. **5**

1.1 The Problem: Modeling Joint Distributions **5**

1.1.1 The Chain Rule 5

1.2 Structural Assumptions **6**

1.2.1 Bayesian Networks 6

1.2.2 The Markov Assumption 6

1.2.3 Why Structure Alone Is Not Enough 7

1.3 Parametric Modeling of Conditional Distributions **7**

1.4 Autoregressive Models **8**

1.5 Fixed-Length Autoregressive Models **9**

1.5.1 FVSBN 10

1.5.2 NADE: Neural Autoregressive Distribution Estimators 12

1.5.3 Autoencoders: A Necessary Digression 15

1.5.4 MADE: Masked Autoencoder for Distribution Estimation 17

1.5.5 PixelRNN and PixelCNN: Spatial Structure in Images 21

1.5.6 WaveNet: Dilated Convolutions for Audio 21

1.5.7 The Fixed-Length Ceiling 22

1.5.8 Summary: Fixed-Length Models 22

1.6 Variable-Length Autoregressive Models **22**

1.6.1 Machine Translation: From Images to Language 22

1.6.2 Words to vectors 23

1.6.3 RNNs: The Hidden State as Context 31

1.6.4 LSTM and GRU: Fixing the Memory 34

1.6.5 Attention 37

1.6.6 Transformer 44

1.6.7 Summary: Variable-Length Models 59

PART I

Explicit Likelihood Models

These models define a tractable probability distribution $p_\theta(x)$ explicitly and train by maximizing the log-likelihood of the data. The key question in each chapter is: how do we make the joint distribution over high-dimensional data both expressive and computationally feasible?

Chapters in this part:

Chapter 1	Autoregressive Models
Chapter 2	Normalizing Flows *(coming soon)*
Chapter 3	Variational Autoencoders *(coming soon)*
Chapter 4	Classical Latent Variable Models *(coming soon)*

Chapter 1. Autoregressive Models

Fully Visible Models: Fixed-Length and Variable-Length

Slides (Google Drive) Video (YouTube)

1.1 The Problem: Modeling Joint Distributions

An image can be viewed as a collection of random variables, one per pixel. More generally, any observation can be represented as a vector $(X_1, X_2, \ldots, X_n)$.
Generative modeling asks a single question:

> *How do we model the joint distribution $p(X_1, X_2, \ldots, X_n)$?*

If we can model this distribution well, we can:

- sample new data,
- condition on partial observations or prompts,
- infer hidden structure from raw inputs.

For simplicity, we assume binary variables in this chapter. (The same ideas extend to discrete or continuous settings.)
A direct approach is to represent the joint distribution explicitly as a **lookup table**: for every possible configuration of $(X_1, \ldots, X_n)$, we store its probability.
If the variables are binary, there are 2^n possible configurations. Since probabilities must sum to one, the number of independent parameters is

$$2^n - 1.$$

Thus the parameter complexity grows exponentially with n. For high-dimensional data such as images, this representation is computationally impossible.
The central challenge of generative modeling is therefore:

> *How can we represent the joint distribution without exponential cost?*

1.1.1 The Chain Rule

A natural first step is to rewrite the joint distribution using the **probability chain rule**:

$$p(X_1, \ldots, X_n) = \prod_{i=1}^{n} p(X_i \mid X_1, \ldots, X_{i-1}).$$

This decomposition is always exact — no assumptions are introduced.

However, decomposition is not compression.

If each conditional distribution is represented as a lookup table, then for binary variables the total number of parameters becomes

$$1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1,$$

which is still exponential in n.

For example, a modest 32×32 binary image would require 2^{1024} entries — clearly infeasible.

The chain rule changes the form of the problem but not its complexity. To make progress, we must introduce additional structure.

1.2 Structural Assumptions

The exponential cost arises because each variable is conditioned on *all* previous variables. In many real systems, however, only a small subset of those dependencies may actually matter.

This motivates a first strategy:

> *Reduce complexity by restricting which variables each conditional depends on.*

1.2.1 Bayesian Networks

Definition 1.1: Bayesian Network

A **Bayesian network** is a directed graphical model in which the joint distribution factorizes as

$$p(X_1, \ldots, X_n) = \prod_{i=1}^{n} p(X_i \mid \text{Parents}(X_i)).$$

Each variable depends only on a selected subset of variables called its **parents**.

If each variable has at most k parents and variables are binary, each conditional table has 2^k entries. The total parameter count becomes

$$O(n \cdot 2^k),$$

which is linear in n when k is small.

Structural assumptions reduce complexity by limiting dependencies.

1.2.2 The Markov Assumption

We can push this idea to its extreme by assuming each variable depends only on the immediately preceding one.

Definition 1.2: First-Order Markov Assumption

The **first-order Markov assumption** states

$$X_{i+1} \perp \{X_1, \ldots, X_{i-1}\} \mid X_i.$$

The resulting joint distribution becomes

$$p(X_{1:n}) = p(X_1) \prod_{i=1}^{n-1} p(X_{i+1} \mid X_i).$$

Here each variable has only one parent, so only local transition probabilities are required.

The Markov assumption is a modeling choice: we intentionally restrict dependencies to obtain tractability.

Both Bayesian networks and Markov chains follow the same philosophy: reduce complexity by narrowing *what we condition on*.

1.2.3 Why Structure Alone Is Not Enough

Structural assumptions help, but they do not fully solve the problem.

Even when dependencies are sparse, lookup tables scale exponentially with the size of the conditioning context. If the parent set becomes large, tabular representations again become intractable.

This suggests a second, fundamentally different strategy.

1.3 Parametric Modeling of Conditional Distributions

Instead of reducing dependencies, we can change how conditionals are represented.

Lookup tables memorize a probability for every possible configuration. But we can instead *compute* probabilities using a parameterized function.

Definition 1.3: Parametric Conditional Model

A **parametric conditional model** represents conditional probabilities using a parameterized function:

$$p(X_i \mid X_{<i}) = f_{\theta_i}(X_{<i}),$$

where the function outputs a valid probability distribution. The parameters may be independent for each conditional or shared across conditionals.

The key difference is conceptual:

- Lookup tables memorize each context independently.
- Functions share parameters and generalize across contexts.

Complexity is now controlled by the function class rather than by the number of possible

inputs.

This allows us to keep the full chain-rule factorization while avoiding exponential storage.

Generative models become tractable through two complementary design dimensions.

1. Structural assumptions simplify the dependency structure by restricting which variables each conditional distribution depends on:

Approach	Dependency	Parameters
Raw chain rule	All previous variables	$O(2^n)$
Bayesian network	Small parent set (k parents)	$O(n \cdot 2^k)$
Markov chain	Previous variable only	$O(n)$

2. Parametric modeling simplifies representation by replacing explicit probability tables with parameterized functions:

- Tabular models store probabilities independently for each context.
- Parametric models compute probabilities using shared parameters.

Structural assumptions control *which dependencies are modeled*, while parametric modeling controls *how conditional distributions are represented*.

These two dimensions are complementary rather than mutually exclusive. Modern generative models often combine both: structural assumptions define the factorization of the joint distribution, while parametric function approximation makes high-dimensional conditionals tractable.

1.4 Autoregressive Models

The name **autoregressive** originates from classical time-series analysis and is composed of two parts:

- **Regression** refers to predicting a variable from other variables. In statistics, regression simply means modeling a quantity as a function of observed inputs.
- **Auto** means *self*. In an autoregressive model, the inputs used for prediction come from the same sequence being modeled.

Combining the two ideas, an autoregressive model performs *self-regression*: each variable is predicted from previously observed variables in the same sequence. Conceptually, the model learns how the past explains the future.

Formally, this idea is expressed by modeling each element x_i conditioned on the preceding elements $x_{<i}$.

Definition 1.4: Autoregressive Model

An **autoregressive model** applies the chain rule factorization exactly:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1})$$

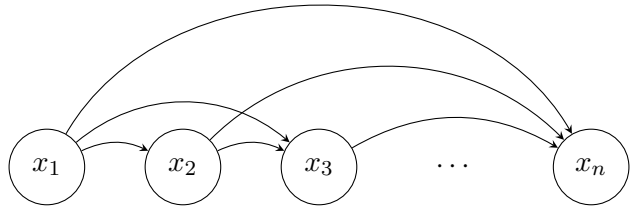

Figure 1.1: Autoregressive model with full past dependencies

but represents each conditional with a **learned function** f_{θ_i} rather than a lookup table. No independence assumptions are introduced — the full dependency structure of the chain rule is preserved.

Sampling proceeds sequentially:

1. Sample $x_1 \sim p(x_1)$
2. Sample $x_2 \sim f_{\theta_2}(x_1)$
3. Sample $x_3 \sim f_{\theta_3}(x_1, x_2)$
4. Continue until x_n

By choosing a specific function class, we implicitly restrict which distributions the model can represent. But this is a far more flexible restriction than hard independence assumptions — and it scales gracefully as the function class becomes more expressive.

Example

Consider a sequence of binary variables x_1, x_2, x_3 with learned conditionals:

$$p(x_1 = 1) = 0.6$$
$$p(x_2 = 1 \mid x_1) = f_{\theta_2}(x_1)$$
$$p(x_3 = 1 \mid x_1, x_2) = f_{\theta_3}(x_1, x_2)$$

To sample a sequence:

1. Draw x_1: get $x_1 = 1$ with probability 0.6
2. Feed x_1 into f_{θ_2} to obtain $p(x_2 = 1 \mid x_1)$, then sample x_2
3. Feed (x_1, x_2) into f_{θ_3} to obtain $p(x_3 = 1 \mid x_1, x_2)$, then sample x_3

At no point do we enumerate all $2^3 = 8$ configurations. The function does the work.

1.5 Fixed-Length Autoregressive Models

We said that instead of storing $p(x_i \mid x_1, \ldots, x_{i-1})$ as a lookup table, we could *compute* it using a learned function f_θ. But we never said what f_{θ_i} actually looks like.

This chapter builds up the answer — starting from the simplest possible parameterization

and progressively replacing its weaknesses with better ideas.

Section 1.5 covers fixed-length models. Throughout, we use MNIST as our running example: $28 \times 28 = 784$ binary pixels in raster scan order, $x_1, \ldots, x_{784} \in \{0, 1\}$.

1.5.1 FVSBN

A common and conceptually simple choice is to model each conditional distribution using logistic regression. In this setting, the conditional probability is parameterized as

$$p_\theta(x_i = 1 \mid x_{<i}) = \sigma(W_i x_{<i} + b_i)\,,$$

where $\sigma(\cdot)$ denotes the logistic sigmoid function. so each variable is predicted using a sigmoid applied to a linear function of the previous variables.. The name **Fully Visible Sigmoid Belief Network (FVSBN)** reflects three properties: the model defines probability directly over the observed variables (fully visible), each conditional distribution is parameterized with a sigmoid (logistic regression), and the joint distribution is factorized as a directed probabilistic model — historically called a belief network.

For a conditional depending on m binary variables, a lookup table requires $2^m - 1$ independent parameters. If the conditional is instead represented by a parameterized function, such as a logistic function, the number of parameters grows only linearly with m. For example, a linear logistic parameterization requires $m + 1$ parameters. The reduction in complexity comes from restricting the space of possible conditional distributions to those expressible by the chosen function.

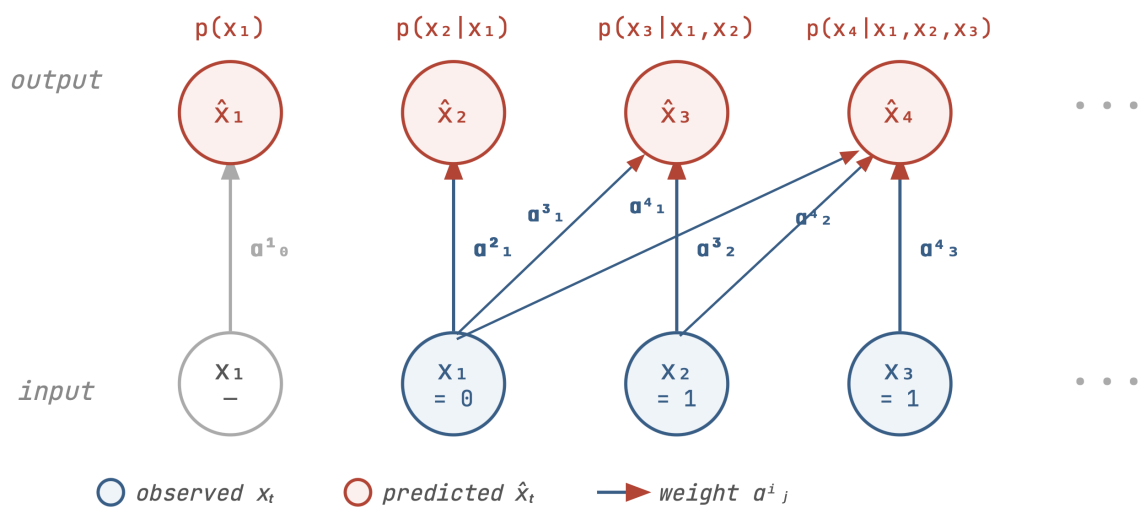

Figure 1.2: Illustration of FVSBN architecture

Definition 1.4: Fully Visible Sigmoid Belief Network (FVSBN)

A **Fully Visible Sigmoid Belief Network** parameterizes each conditional as:

$$\hat{x}_i = p(x_i = 1 \mid x_{<i};\, \alpha^i) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i\, x_j\right)$$

where $\sigma(z) = 1/(1+e^{-z})$ and α^i are learned parameters for position i. The joint:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_{<i})$$

Parameters: $1 + 2 + \cdots + n = O(n^2/2)$.

Example: Evaluating and Sampling from FVSBN

Define the conditional probabilities using the parameter matrix

$$\hat{x}_i = p(x_i = 1 \mid x_{<i};\, \alpha^i) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i\, x_j\right).$$

Collect the parameters into a lower–triangular matrix

$$A = \begin{bmatrix} \alpha_0^1 & 0 & 0 & 0 \\ \alpha_0^2 & \alpha_1^2 & 0 & 0 \\ \alpha_0^3 & \alpha_1^3 & \alpha_2^3 & 0 \\ \alpha_0^4 & \alpha_1^4 & \alpha_2^4 & \alpha_3^4 \end{bmatrix}, \qquad \tilde{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

Then the vector of conditional probabilities can be written compactly as

$$\hat{\mathbf{x}} = \sigma(A\tilde{x}),$$

where the sigmoid function is applied element-wise.
To evaluate $p(X_1 = 0,\, X_2 = 1,\, X_3 = 1,\, X_4 = 0)$:

1. Compute all $\hat{x}_i$ using the matrix form above.
2. Multiply: $(1 - \hat{x}_1) \times \hat{x}_2 \times \hat{x}_3 \times (1 - \hat{x}_4)$

To sample:

1. Sample $x_1 \sim \text{Bernoulli}(\hat{x}_1)$
2. Sample $x_2 \sim \text{Bernoulli}(\hat{x}_2(x_1))$, and so on

The asymmetry of autoregressive models: Autoregressive models separate evaluation and generation in an important way. Given a known datapoint, the likelihood $p(x)$ can often be computed efficiently, since the conditional terms $p(x_i \mid x_{<i})$ are evaluated with inputs that

are already known. In contrast, sampling is inherently sequential: each variable must be generated before later conditionals can be evaluated. This difference between efficient likelihood evaluation and sequential generation is a defining property of autoregressive modeling.

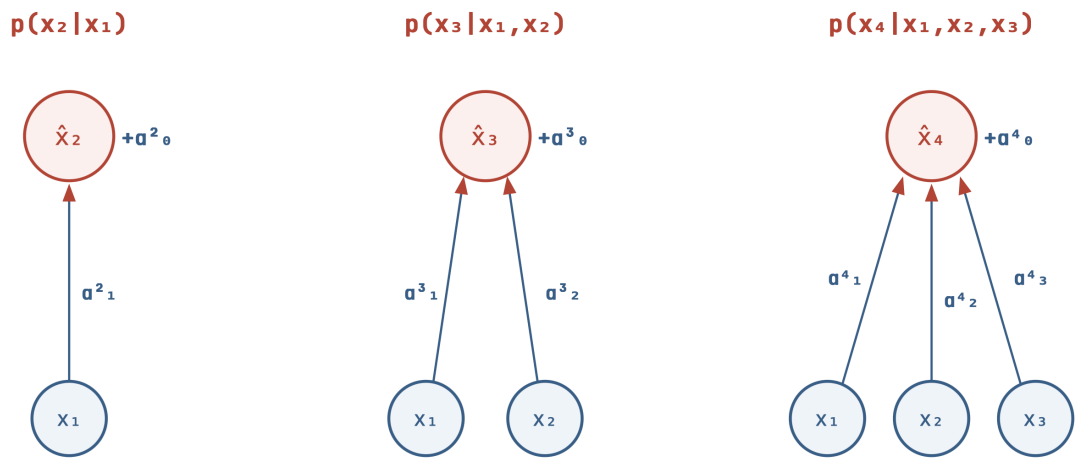

Figure 1.3: Conditionals as logistic regression

Let the conditional distribution be parameterized by a function

$$f_{\theta_i}(x_{<i}) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i \, x_j\right),$$

which is simply a logistic regression model for variable x_i.
Then each conditional can be written compactly as

$$p(x_i = 1 \mid x_{<i}) = f_{\theta_i}(x_{<i}).$$

Using the autoregressive factorization, the joint distribution becomes

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_{<i}) = \prod_{i=1}^{n} f_{\theta_i}(x_{<i}).$$

FVSBN is valid but limited, since the function f_θ is a logistic regression and therefore models only linear dependencies. To obtain richer, more expressive conditionals, we next replace f_θ with neural networks.

1.5.2 NADE: Neural Autoregressive Distribution Estimators

NADE strengthens autoregressive modeling by replacing the linear conditional predictor of FVSBN with a one-hidden-layer neural network. For each position i, a hidden representation is computed and then mapped to a conditional probability:

$$h_i = \sigma\big(A^i x_{<i} + c^i\big), \qquad \hat{x}_i = \sigma\big(\alpha^i h_i + b_i\big).$$

The hidden layer allows the model to capture nonlinear interactions among observed variables, something a purely linear predictor cannot do.

The parameter-scaling problem. Assigning a separate weight matrix $A^i \in \mathbb{R}^{d\times(i-1)}$ to every conditional is costly. Summing over all positions gives

$$\sum_{i=1}^{n} d(i-1) = d\,\frac{n(n-1)}{2} = O(n^2 d)$$

parameters for the input-to-hidden mappings alone. Beyond the raw count, this design is statistically wasteful: the inputs to adjacent conditionals differ by only a single observation, yet each position learns entirely independent features. There is no mechanism for the model to transfer what it learns about earlier variables to later ones.

Weight sharing as the key idea. NADE resolves this by tying the input-to-hidden weights across all positions. A single shared matrix

$$W \in \mathbb{R}^{d\times n}$$

replaces the family $\{A^i\}$, and the i-th conditional uses only the first $i-1$ columns:

$$h_i = \sigma(W_{:,<i}\, x_{<i} + c)\,, \qquad \hat{x}_i = p(x_i = 1 \mid x_{<i}) = \sigma\big(\alpha^i h_i + b_i\big)\,.$$

Because the same column $W_{:,j}$ appears in every conditional that conditions on x_j, the model is forced to learn features that are useful across the entire sequence, not just at a single position. The input-to-hidden parameter count drops to $dn = O(nd)$, and the shared structure enables an efficient left-to-right recurrence: the pre-activation at position $i+1$ is obtained by augmenting the pre-activation at position i with a single rank-one update, rather than recomputing from scratch.

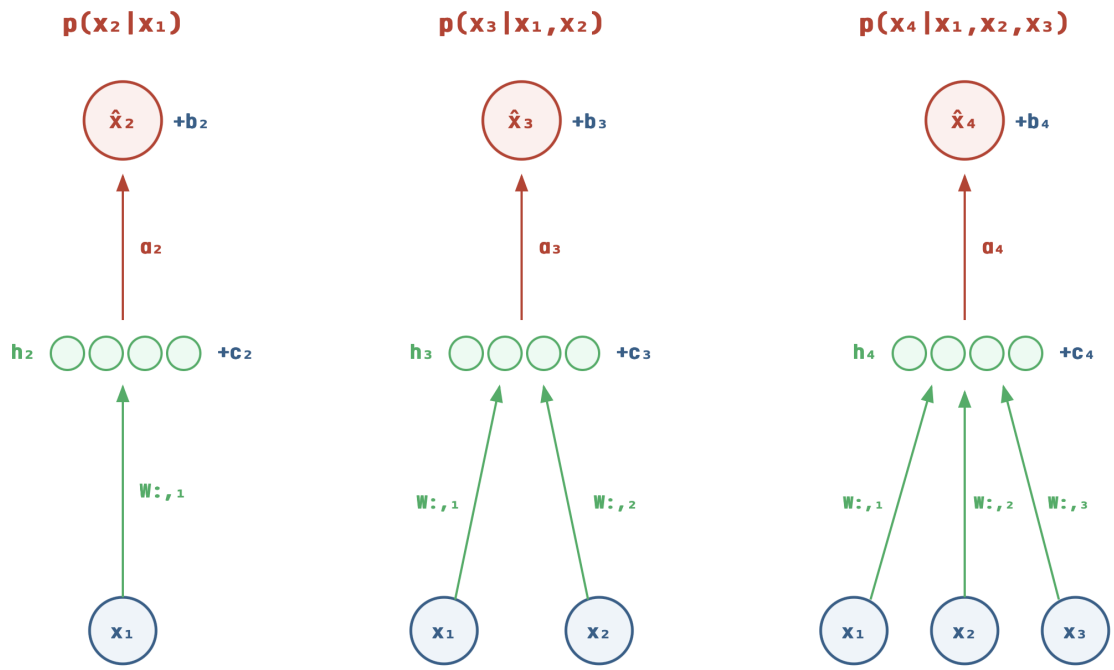

Figure 1.4: Conditionals modeled with h layer.

Definition 1.5: Neural Autoregressive Distribution Estimator (NADE)

NADE learns a single shared weight matrix $W \in \mathbb{R}^{d \times n}$ so that every conditional reuses the same input-to-hidden mapping:

$$h_i = \sigma(W_{:,<i}\, x_{<i} + c_i)\,, \qquad \hat{x}_i = p(x_i = 1 \mid x_{<i}) = \sigma\big(\alpha^i h_i + b_i\big)\,.$$

Columns of W are shared across positions, so hidden pre-activations can be updated incrementally from left to right. Compared with unshared FVSBN-style networks, NADE reduces input-to-hidden parameter count from $O(n^2 d)$ to $O(nd)$ while retaining nonlinear expressiveness.

Extensions. The NADE framework extends naturally beyond binary variables by modifying the output distribution while retaining the same autoregressive hidden representation.

Discrete variables. For non-binary discrete random variables $x_i \in \{1, \ldots, K\}$ (e.g., pixel intensities ranging from 0 to 255), the output $\hat{x}_i$ parameterizes a categorical distribution rather than a Bernoulli. The hidden representation is computed as

$$h_i = \sigma(W_{\cdot,<i} x_{<i} + c_i),$$

and the conditional distribution is modeled as

$$p(x_i \mid x_{<i}) = \mathrm{Cat}(p_i^1, \ldots, p_i^K),$$

where

$$\hat{x}_i = (p_i^1, \ldots, p_i^K) = \mathrm{softmax}(A_i h_i + b_i).$$

The softmax function generalizes the sigmoid by transforming a vector of real-valued scores into a valid probability distribution over K outcomes, ensuring non-negativity and unit normalization.

Continuous variables (RNADE). For real-valued variables $x_i \in \mathbb{R}$ (e.g., speech signals), each conditional can be modeled using a continuous distribution. A common choice is a mixture of Gaussians, giving rise to the Real-valued NADE (RNADE). In this case, the conditional distribution takes the form

$$p(x_i \mid x_{<i}) = \sum_{j=1}^{K} \pi_i^j \, \mathcal{N}\Big(x_i; \mu_i^j, \sigma_i^j\Big),$$

where π_i^j are mixture weights and (μ_i^j, σ_i^j) are the mean and standard deviation of the j-th Gaussian component. These parameters are predicted from the hidden state:

$$h_i = \sigma(W_{\cdot,<i} x_{<i} + c_i), \qquad \hat{x}_i = (\pi_i^1, \ldots, \pi_i^K, \mu_i^1, \ldots, \mu_i^K, \sigma_i^1, \ldots, \sigma_i^K) = f(h_i).$$

In practice, the standard deviations are typically parameterized using an exponential transform to ensure $\sigma_i^j > 0$. This extension allows NADE to model complex continuous densities while preserving the autoregressive factorization and efficient shared-parameter structure.

Remark: Hidden Units as Deterministic Features

Although h_i is commonly called a *hidden layer*, it is more precise to regard it as a **deterministic feature vector** derived from the observations. Given $x_{<i}$, the value of h_i is fixed:

$$h_i = \sigma(W_{:,<i}\, x_{<i} + c_i)\,.$$

NADE therefore contains no latent random variables. The h_i serve purely as nonlinear summaries of the observed context, used to parameterize each conditional distribution.

1.5.3 Autoencoders: A Necessary Digression

Before MADE, we need to understand autoencoders — and why they cannot be used as generative models without modification.

Definition 1.6: Autoencoder

An **autoencoder** maps input x through an encoder f_{enc} to a representation z, then through a decoder f_{dec} to a reconstruction $\hat{x}$:

$$z = f_{\text{enc}}(x), \qquad \hat{x} = f_{\text{dec}}(z), \qquad \mathcal{L} = \sum_{x \in \mathcal{D}} \sum_i [-x_i \log \hat{x}_i - (1 - x_i) \log(1 - \hat{x}_i)]$$

The bottleneck at z forces the network to learn a compressed representation. But a vanilla autoencoder is **not** a generative model: it does not define a distribution over x we can sample from.

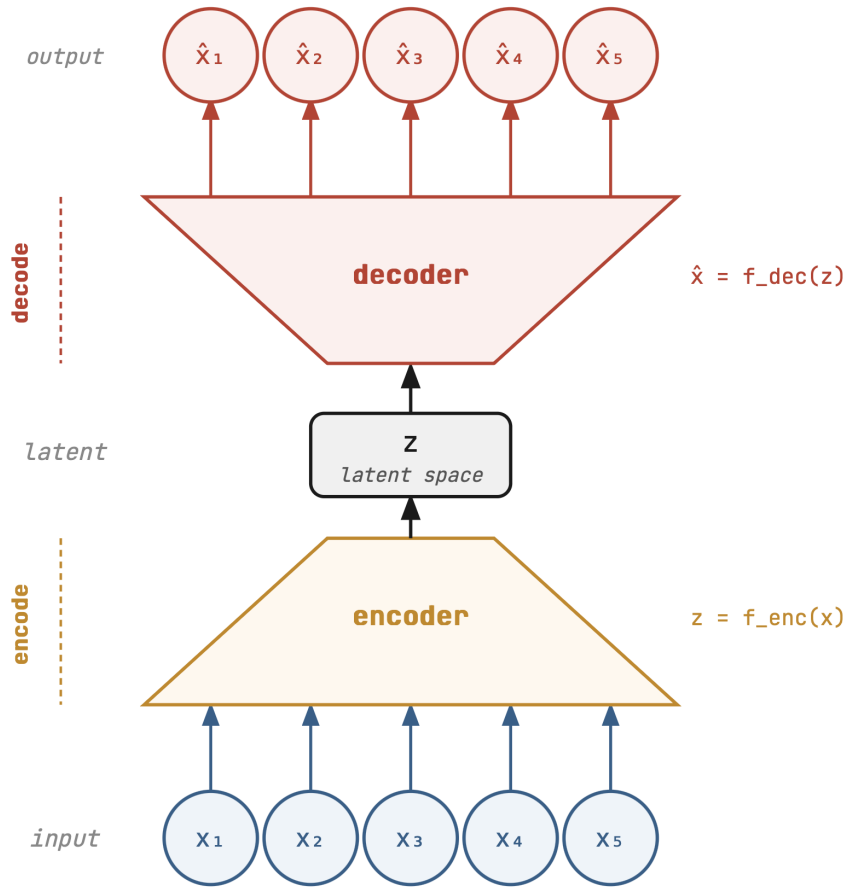

Figure 1.5: Autoencoder

On the surface, FVSBN and NADE already look like autoencoders: they take x as input and produce $\hat{x}$ with the same cross-entropy loss. The problem is that a standard autoencoder lets output $\hat{x}_i$ see input x_i directly — it can simply copy it, achieving zero reconstruction error while learning nothing useful.

The cheating problem: Output $\hat{x}_i$ must depend *only* on $x_1, \ldots, x_{i-1}$ to be useful as a generative model. At generation time we do not have x_i to copy — that is what we are trying to produce. A standard autoencoder does not enforce this constraint.

1.5.4 MADE: Masked Autoencoder for Distribution Estimation

An ordinary feed-forward autoencoder can be transformed into an autoregressive density estimator by masking its weight matrices. The goal is to preserve the autoregressive factorization

$$p(x) = \prod_{i=1}^{n} p(x_i \mid x_{<i}),$$

while computing all conditionals in a **single forward pass**.

Challenge. A standard autoencoder is fully connected, meaning each output $\hat{x}_i$ depends on all inputs. This violates the autoregressive constraint, which requires $\hat{x}_i$ to depend only on variables preceding x_i in a chosen ordering.

Key idea: masking as a structural constraint. MADE enforces a directed acyclic graph (DAG) inside the autoencoder by introducing binary masks on the weight matrices. These masks remove invalid connections so that each output obeys autoregressive dependencies. Given an ordering (or permutation) π of the variables, the joint distribution factorizes as

$$p(x) = \prod_{i=1}^{n} p(x_{\pi_i} \mid x_{\pi_{<i}}).$$

For example, if $\pi = (x_2, x_3, x_1)$, then

$$p(x_1, x_2, x_3) = p(x_2)\, p(x_3 \mid x_2)\, p(x_1 \mid x_2, x_3).$$

The output unit producing $\hat{x}_2$ is therefore not allowed to depend on any input, while the unit producing $p(x_3 \mid x_2)$ may depend only on x_2, and so forth.

Hidden-unit degrees. Instead of manually designing masks, MADE assigns each hidden unit k an integer degree

$$m(k) \in \{1, \ldots, n-1\}.$$

The degree specifies the largest input index that the hidden unit is allowed to depend on. Intuitively, it determines how far into the input ordering the unit can "see".

Mask construction. Masks are constructed using these degrees:

- Input $\to$ hidden: a hidden unit with degree $m(k)$ connects only to inputs with index $j \leq m(k)$.
- Hidden $\to$ hidden: connections satisfy

  $$m(\text{prev}) \leq m(\text{next}),$$

  ensuring autoregressive consistency across layers.

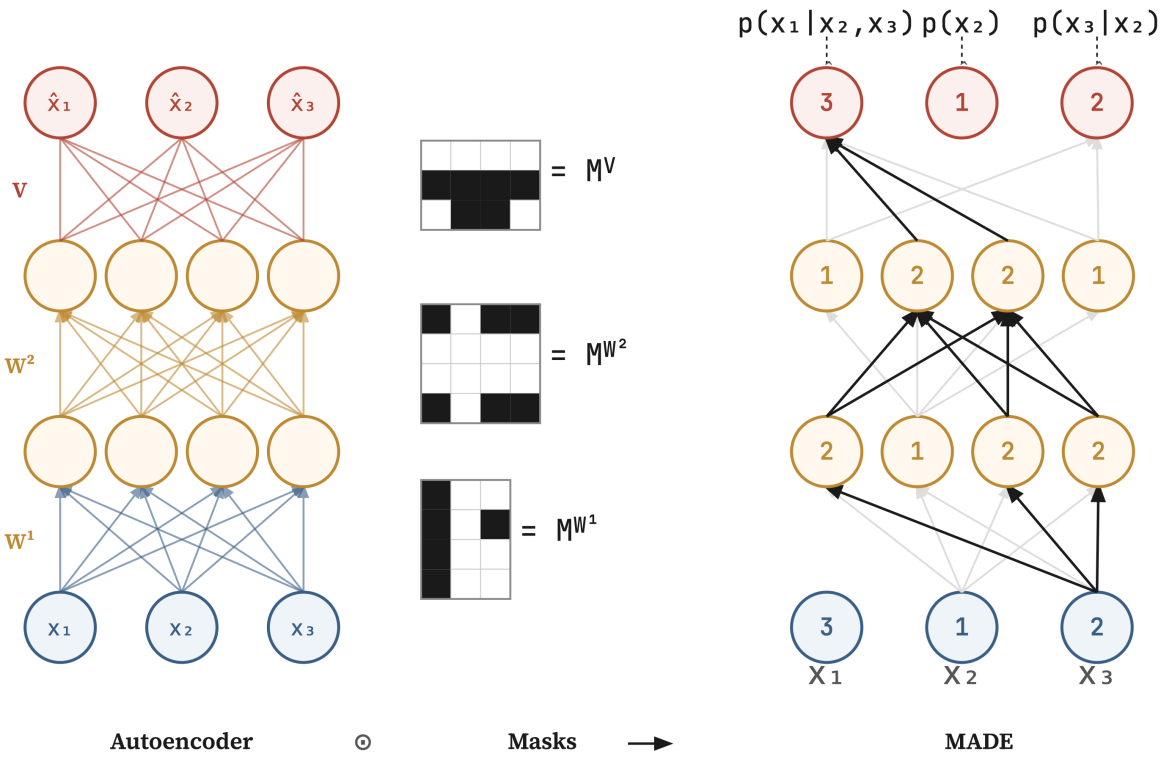

Figure 1.6: Applying masks to Autoencoder to get MADE. Connections in light gray correspond to paths that depend only on 1 input, while the dark gray connections depend on 2 inputs.

- Hidden $\to$ output: output $\hat{x}_i$ receives connections only from hidden units with degree

$$m < i.$$

These constraints guarantee that

$$\hat{x}_i \text{ depends only on } x_{<i},$$

so the network represents a valid autoregressive model.
Here is the paragraph you can add to introduce the masked forward pass equations:

Masked forward pass. Once the degrees are assigned and the masks are constructed, the autoregressive constraints are baked directly into the network weights via the **Hadamard product** (element-wise multiplication, denoted $\odot$). For a single hidden layer, the masked forward pass is written as

$$h(x) = g\big(b + (W \odot M^W)\, x\big)\,,$$
$$\hat{x} = \sigma\big(c + (V \odot M^V)\, h(x)\big)\,,$$

where g is a nonlinear activation function, σ denotes the sigmoid, b and c are bias vectors, and W, V are the weight matrices for the input-to-hidden and hidden-to-output layers respectively.

The binary matrices M^W and M^V are the masks derived from the degree assignments described above: an entry of $M^W_{k,j} = 1$ if and only if $m(k) \geq j$, and an entry of $M^V_{i,k} = 1$ if and only if $m(k) < i$. By zeroing out the corresponding entries in the weight matrices, the Hadamard product enforces the connectivity constraints structurally, so that no gradient or information can flow through a forbidden path. The entire responsibility of satisfying the autoregressive property therefore rests with M^W and M^V, and the remaining parameters W, V, b, c are free to be optimized by standard backpropagation without any additional constraints.

Computational structure. NADE and MADE both exploit shared computations, but differ in how the autoregressive constraint is enforced. NADE computes conditionals through a left-to-right recurrence that incrementally updates hidden pre-activations across positions. MADE instead encodes the same autoregressive dependencies directly in the network connectivity via masks, allowing all outputs to be produced by a single masked forward pass without an explicit sequential recurrence.

Order-agnostic training. Autoregressive models usually assume a fixed ordering of variables (e.g., raster scan for images). This introduces an artificial bias, since variables appearing early in the order cannot depend on later ones.

MADE removes this limitation by randomly changing the variable ordering during training and regenerating the corresponding masks. Each ordering defines a different valid autoregressive factorization of the same joint distribution.

Training across many orderings reduces reliance on any single ordering and encourages the model to learn more robust dependencies. This can be viewed as implicitly training an ensemble of autoregressive models while sharing the same parameters.

An additional benefit is flexibility: by choosing an appropriate ordering, inference tasks such as imputing missing values become easier.

At sampling time, however, a single ordering is chosen, and variables are generated sequentially according to that order.

Definition 1.7: Masked Autoencoder for Distribution Estimation (MADE)

MADE is an autoregressive model obtained by applying binary masks to the weight matrices of an autoencoder. Hidden units are assigned ordering degrees that determine allowed connections, ensuring that each output $\hat{x}_i$ depends only on preceding variables. This allows all conditional distributions to be computed in a single forward pass while preserving autoregressive structure.

Example: MADE with ordering $x_2 \to x_3 \to x_1$

Consider a binary vector

$$x = (x_1, x_2, x_3), \qquad x_i \in \{0, 1\},$$

and assume the autoregressive ordering

$$x_2 \to x_3 \to x_1.$$

Autoregressive factorization. The joint distribution becomes

$$p(x) = p(x_2)\, p(x_3 \mid x_2)\, p(x_1 \mid x_2, x_3).$$

Training (stochastic gradient descent). Given a training example (x_1, x_2, x_3), MADE receives the full binary vector as input. Due to masking, the outputs represent the conditionals:

$$\hat{x}_2 = p(x_2), \qquad \hat{x}_3 = p(x_3 \mid x_2), \qquad \hat{x}_1 = p(x_1 \mid x_2, x_3).$$

Each output is a Bernoulli probability rather than a reconstructed binary value. Parameters are updated using stochastic gradient descent by minimizing the negative log-likelihood (binary cross-entropy).

Sampling (generation). After training, sampling follows the chosen ordering, requiring one forward pass per variable. We trace through each pass below.

Forward Pass 1. The input is zero-initialised since nothing has been sampled yet:

$$\text{input} = [x_1, x_2, x_3] = [0,\ 0,\ 0].$$

MADE runs forward. By construction of the masks, the output $\hat{x}_2$ has no dependencies. We read off $p(x_2)$ and sample

$$x_2 \sim \text{Bernoulli}(p(x_2)), \qquad \text{e.g. } x_2 = 1.$$

Forward Pass 2. The sampled value of x_2 is inserted into the input:

$$\text{input} = [0,\ 1,\ 0].$$

MADE runs forward again. The masks now allow $\hat{x}_3$ to attend to x_2. We read off $p(x_3 \mid x_2 = 1)$ and sample

$$x_3 \sim \text{Bernoulli}(p(x_3 \mid x_2 = 1)), \qquad \text{e.g. } x_3 = 0.$$

Forward Pass 3. Both sampled values are inserted into the input:

$$\text{input} = [0,\ 1,\ 0].$$

MADE runs forward a final time. The masks allow $\hat{x}_1$ to attend to both x_2 and x_3. We read off $p(x_1 \mid x_2 = 1,\ x_3 = 0)$ and sample

$$x_1 \sim \text{Bernoulli}(p(x_1 \mid x_2 = 1,\ x_3 = 0)), \qquad \text{e.g. } x_1 = 1.$$

The final sampled tuple is

$$(x_1, x_2, x_3) = (1, 1, 0),$$

which forms a new generated example. Although MADE computes all conditionals in one forward pass during training, generation requires D forward passes (one per variable) because each newly sampled variable changes the conditioning context. Only the output corresponding to the next variable in the ordering is read at each step; the remaining outputs are discarded.

1.5.5 PixelRNN and PixelCNN: Spatial Structure in Images

FVSBN, NADE, and MADE treat an image as a flat sequence, ignoring two-dimensional spatial structure. Real images also have pixel values in $\{0, \ldots, 255\}$ across three colour channels. For each pixel x_t, the channels are themselves factorized:

$$p(x_t \mid x_{<t}) = p(x_t^{\text{red}} \mid x_{<t})\, p(x_t^{\text{green}} \mid x_{<t}, x_t^{\text{red}})\, p(x_t^{\text{blue}} \mid x_{<t}, x_t^{\text{red}}, x_t^{\text{green}})$$

Each channel conditional is a categorical distribution over 256 values.

Definition 1.8: PixelRNN

PixelRNN processes pixels in raster scan order using LSTM variants with masking to preserve the autoregressive ordering. Each pixel's hidden state h_t is a *deterministic* function of all previously observed pixels.

- Captures long-range spatial dependencies via the recurrent hidden state
- Exact tractable likelihood
- Sequential evaluation — cannot be parallelized, very slow to train
- **Fully visible:** h_t is deterministic, not a stochastic latent variable

Definition 1.9: PixelCNN

PixelCNN replaces recurrence with masked convolutions. Each convolutional filter is zeroed out for all pixels that follow the current position in raster scan order. Stacking masked convolutional layers grows the receptive field while keeping computation parallelizable.

- Exploits spatial locality via convolutions
- Likelihood evaluation is fully parallelizable during training
- Generation remains sequential
- Similar sample quality to PixelRNN, significantly faster to train
- **Fully visible:** all feature maps are deterministic functions of observed pixels

PixelCNN's parallelizable training led directly to **PixelDefend** — a defence against adversarial examples that trains a PixelCNN on clean data and flags inputs with anomalously low $p(x)$.

1.5.6 WaveNet: Dilated Convolutions for Audio

The masked convolution idea extends to raw audio. Audio sampled at 16kHz has 16,000 timesteps per second, with dependencies spanning thousands of steps — far beyond a standard convolutional filter.

Definition 1.10: Dilated Convolutions (WaveNet)

A **dilated convolution** with dilation d skips $d-1$ samples between each filter tap. Stacking layers with exponentially increasing dilations $d = 1, 2, 4, \ldots, 2^{L-1}$ gives an exponentially growing receptive field:

$$\text{receptive field} = 2^L$$

Fully parallelizable during training. Generation remains sequential.

Fully visible: all computations are deterministic functions of observed audio samples.

1.5.7 The Fixed-Length Ceiling

All models in this section assume sequence length is fixed at design time. The mask is built for n variables. The architecture has n outputs.

This works for images — 28×28 images always have 784 pixels. It fails completely for language: sentences have different lengths, and a model needs to handle sequences of length 3 and sequences of length 3000 with the same parameters.

This is the boundary between the two sections. Section 2.2 introduces the hidden state h as a compressed, dynamically updated context that frees the model from knowing n in advance.

1.5.8 Summary: Fixed-Length Models

Model	Key idea	Parameters	Parallel training
FVSBN	Logistic regression per position	$O(n^2)$	✓
NADE	Shared weights, n passes	$O(nd)$	✓
MADE	Masking, single pass	$O(nd)$	✓
PixelRNN	LSTM over pixels	$O(nd^2)$	×
PixelCNN	Masked convolutions	$O(nd^2)$	✓
WaveNet	Dilated convolutions for audio	$O(nd^2)$	✓

All are fully visible — exact likelihood, no latent variables, no natural way to extract features. Section 2.2 shows what happens when we lift the fixed-length assumption.

1.6 Variable-Length Autoregressive Models

1.6.1 Machine Translation: From Images to Language

Every model in the previous section — FVSBN, NADE, MADE, PixelRNN, PixelCNN, WaveNet — was designed for data with a **fixed, known dimensionality**. A 28×28 image always has 784 pixels. The mask is built for exactly n variables. The architecture has exactly n outputs. This is not a limitation we can engineer around; it is baked into the structure of these models.

Language breaks this assumption completely. A sentence can be 3 words or 3000 words. A paragraph, a document, a codebase — none of these have a fixed length known at design time.

We cannot build a mask for n variables if we do not know n.

But the problem runs deeper than variable length. With pixels, the input space is natural: each pixel $x_i \in \{0, \ldots, 255\}$ is already a number. Language does not come with numbers attached. Before we can model sequences of words, we must first answer a more fundamental question:

> *How do we represent a word as a mathematical object?*

1.6.2 Words to vectors

Terminology: Types and Tokens

Before discussing representations, we fix terminology.

Word Types and Tokens

A **word type** is an element of a finite vocabulary $\mathcal{V}$ — a word in the abstract, independent of context. A **word token** is a specific instance of a type as observed in context.

Most word embedding methods produce a single representation per *type*, which is then used for every token of that type regardless of context.

The Vocabulary and the One-Hot Representation

Fix a vocabulary $\mathcal{V}$ of size $|\mathcal{V}| = V$ — the set of all distinct tokens the model will ever see. In practice V ranges from tens of thousands to hundreds of thousands.

The simplest representation assigns each word $w \in \mathcal{V}$ a unique integer index $i(w) \in \{1, \ldots, V\}$. From this index we construct a **one-hot vector**:

$$\mathbf{e}_w \in \{0,1\}^V, \qquad (\mathbf{e}_w)_j = \begin{cases} 1 & j = i(w) \\ 0 & \text{otherwise.} \end{cases}$$

Two critical failures of one-hot representations.

No geometry. Every pair of distinct words is equidistant:

$$\|\mathbf{e}_w - \mathbf{e}_{w'}\| = \sqrt{2} \qquad \text{for all } w \neq w'.$$

Equivalently, $\mathbf{e}_{\text{tea}}^\top \mathbf{e}_{\text{coffee}} = \mathbf{e}_{\text{tea}}^\top \mathbf{e}_{\text{the}} = 0$. "King" is as far from "queen" as it is from "cabbage."

No generalization. Any function on one-hot vectors must learn each word's behaviour independently. There is no mechanism to transfer what the model learns about "run" to "running," or from "Paris" to "Rome."

These failures are not minor inconveniences — they mean one-hot representations cannot

support the kind of generalization that language modelling requires.

Word Vectors: The Distributional Hypothesis

The solution is to represent each word as a dense vector in a low-dimensional continuous space:

$$\mathbf{w} \in \mathbb{R}^d, \qquad d \ll V.$$

These are called **word vectors** or **word embeddings**. The theoretical foundation comes from a classic observation in linguistics:

The Distributional Hypothesis *(Firth, 1957)*

"You shall know a word by the company it keeps."

Words that appear in similar contexts tend to have similar meanings. "Coffee" and "tea" both appear near "hot," "drink," "morning," and "cup." A good representation should place them close together in $\mathbb{R}^d$.

The Embedding Matrix. Given a vocabulary of size V and embedding dimension d, the **embedding matrix** is $E \in \mathbb{R}^{d \times V}$. The word vector for word w with index $i(w)$ is the corresponding column:

$$\mathbf{w} = E\,\mathbf{e}_w = E_{:,\,i(w)} \in \mathbb{R}^d.$$

The matrix E is learned during training. Each column is a d-dimensional representation of one word. Typical values: $d \in \{50, 100, 300, 768, \dots\}$ versus $V \in \{30{,}000, \dots, 100{,}000\}$.

The embedding matrix reduces input dimensionality from V (sparse, orthogonal) to d (dense, continuous), and crucially allows the model to generalize across words with similar embeddings.

Word2Vec: Learning Vectors from Context

The question is: how do we learn E? One approach is to learn embeddings jointly with a downstream task. Another is to pre-train them from raw text using a self-supervised objective designed to capture distributional similarity.

Word2Vec (Mikolov et al., 2013) introduced two such objectives.

The Skipgram Model

Skipgram Word2Vec

Maintain two matrices $U, V \in \mathbb{R}^{|\mathcal{V}| \times d}$. Each word w has an **outside vector** $\mathbf{u}_w$ (row of U, used when w is a context word) and a **center vector** $\mathbf{v}_w$ (row of V, used when w is the center word).

Given a center word c, the model probability of outside word o is:

$$p(o \mid c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \mathcal{V}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}.$$

The objective maximizes the log-likelihood of all observed context pairs over a window of size k:

$$\mathcal{L}(U, V) = \sum_{d} \sum_{i=1}^{m} \sum_{j=1}^{k} -\log\, p\Big(w_{i-j}^{(d)} \;\Big|\; w_i^{(d)}\Big)\,.$$

The dot product $\mathbf{u}_o^\top \mathbf{v}_c$ measures the compatibility of o as a context for center word c. Training pushes this dot product up for observed pairs and down for all other words via the softmax normalization.

Gradient Intuition: Observed Minus Expected

Working out $\nabla_{\mathbf{v}_c} \log p(o \mid c)$ yields:

$$\nabla_{\mathbf{v}_c} \log p(o \mid c) \;=\; \mathbf{u}_o \;-\; \sum_{x \in \mathcal{V}} p(x \mid c)\, \mathbf{u}_x \;=\; \mathbf{u}_o - \mathbb{E}_{x \sim p(\cdot \mid c)}[\mathbf{u}_x].$$

> The gradient decomposes into **observed minus expected**: we push $\mathbf{v}_c$ to be more like the actually-observed context word $\mathbf{u}_o$, and less like the average context word the model currently expects. The learning signal is entirely grounded in the distributional hypothesis.

CBOW: The Reverse Objective

Continuous Bag of Words (CBOW) flips the direction: given all surrounding context words, predict the center word. The context vectors are averaged and passed through the same softmax.

	Skipgram	**CBOW**
Training speed	Slower	Faster
Rare words	Better	Worse
Frequent words	Worse	Better
Analogy tasks	Better	Worse

Negative Sampling

The softmax denominator requires summing over all V words at every step — $\mathcal{O}(V)$ per update, prohibitive for large vocabularies. Negative sampling approximates it with a binary classification task:

Skip-Gram with Negative Sampling (SGNS)

Replace the full softmax with:

$$\mathcal{L}_{\text{SGNS}} = \log \sigma\left(\mathbf{u}_o^\top \mathbf{v}_c\right) + \sum_{\ell=1}^{k} \mathbb{E}_{u_\ell \sim p_{\text{neg}}}\left[\log \sigma\left(-\mathbf{u}_\ell^\top \mathbf{v}_c\right)\right],$$

where σ is the logistic function and p_{neg} is a noise distribution (typically the unigram distribution raised to the $\frac{3}{4}$ power).
This reduces each training step from $\mathcal{O}(V)$ to $\mathcal{O}(k)$ with $k \ll V$ (typically $k \in \{5, 15\}$).

Why does negative sampling work? The full partition function pushes down on *every* word other than the observed one. Negative sampling observes that we do not need to push down on all words simultaneously — a small random sample of k "noise" words provides sufficient signal at a fraction of the cost.

The Geometry of Word Vectors

Well-trained word vectors exhibit a striking emergent property: semantic and syntactic relationships are encoded as **linear structure** in the embedding space.

The Analogy Property

The canonical example is:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}.$$

This is not a designed property — it emerges from training on raw text. The offset $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}}$ encodes "royalty independent of gender." Adding $\mathbf{v}_{\text{woman}}$ recovers the female equivalent.

Emergent Linear Structure

Similar patterns arise across many relationship types:

Relationship	**Vector arithmetic**
Verb tense	$\mathbf{v}_{\text{walking}} - \mathbf{v}_{\text{walk}} \approx \mathbf{v}_{\text{swimming}} - \mathbf{v}_{\text{swim}}$
Country–capital	$\mathbf{v}_{\text{Paris}} - \mathbf{v}_{\text{France}} \approx \mathbf{v}_{\text{Berlin}} - \mathbf{v}_{\text{Germany}}$
Comparative	$\mathbf{v}_{\text{bigger}} - \mathbf{v}_{\text{big}} \approx \mathbf{v}_{\text{taller}} - \mathbf{v}_{\text{tall}}$
Superlative	$\mathbf{v}_{\text{worst}} - \mathbf{v}_{\text{bad}} \approx \mathbf{v}_{\text{biggest}} - \mathbf{v}_{\text{big}}$

The linear structure is what makes generalization possible: a model that has seen "Paris is the capital of France" can reason about "Berlin is the capital of Germany" because the relationship is encoded as a consistent geometric offset.

GloVe: Global Co-occurrence Statistics

Word2Vec learns from local context windows. **GloVe** (Global Vectors for Word Representation, Pennington et al., 2014) builds directly on the full co-occurrence statistics of the corpus.

Two Families of Methods

> Prior to GloVe, embedding methods fell into two families. **Count-based** methods (LSA, HAL) efficiently exploit global corpus statistics but produce sub-optimal geometry, performing poorly on analogy tasks. **Window-based** methods (skip-gram, CBOW) capture complex linguistic patterns but process only local context and discard global statistics. GloVe is designed to get the best of both.

The Co-occurrence Matrix

Let X_{ij} denote the number of times word j appears in the context of word i across the entire corpus, and let $P_{ij} = X_{ij}/\sum_k X_{ik}$ be the corresponding probability.

The ratio P_{ik}/P_{jk} for a probe word k carries information about the relationship between i and j: if k is related to i but not j the ratio is large; if related to neither or both, it is near 1. GloVe trains embeddings to capture this ratio structure directly.

The GloVe Objective

GloVe Objective

$$\mathcal{L}_{\text{GloVe}} = \sum_{i,j=1}^{V} f(X_{ij})\Big(\mathbf{v}_i^\top \tilde{\mathbf{v}}_j + b_i + \tilde{b}_j - \log X_{ij}\Big)^2,$$

where $\mathbf{v}_i$, $\tilde{\mathbf{v}}_j$ are word and context vectors, b_i, $\tilde{b}_j$ are scalar biases, and $f(X_{ij})$ is a weighting function that downweights very frequent co-occurrences and ignores zero-count pairs. The model trains only on the nonzero entries of X.

Taking $\log X_{ij}$ rather than X_{ij} directly compresses the scale of co-occurrence counts and produces much more useful geometry. The weighting function f prevents the loss from being dominated by very common word pairs like "*the–of*."

	Word2Vec (skipgram)	**GloVe**
Context	Local windows (online)	Full matrix (batch)
Global statistics	Implicit	Explicit
Corpus passes	Many (stochastic)	One (to build X)
Analogy accuracy	Good	Generally better

Evaluating Word Vectors

Intrinsic Evaluation

Intrinsic Evaluation

Evaluation on a specific, isolated subtask — typically analogy completion or correlation with human similarity judgments — that is fast to compute and does not require training a downstream system. Used primarily for hyperparameter tuning (dimension, window size, corpus).

Requirement: must correlate positively with performance on the final downstream task to be meaningful.

Analogy task. Given $a : b :: c :?$, find:

$$d^* = \operatorname*{argmax}_i \frac{(\mathbf{x}_b - \mathbf{x}_a + \mathbf{x}_c)^\top \mathbf{x}_i}{\|\mathbf{x}_b - \mathbf{x}_a + \mathbf{x}_c\|}.$$

Human similarity correlation. Ask humans to rate word-pair similarity on a fixed scale (e.g. 0–10), then compute Spearman correlation with cosine similarities of the corresponding word vectors.

Caveats for analogy evaluation. Semantic analogies (city–state, capital–country) can fail when multiple valid answers exist, or when the training corpus is dated and capitals have changed. Dimensionality reduction to 2D for visualization loses significant spatial structure; relationships clear in high dimensions may vanish in 2D.

Extrinsic Evaluation

Extrinsic Evaluation

Evaluation of word vectors as a component of a real downstream system (named entity recognition, sentiment analysis, question answering). Slow to compute; hard to isolate which subsystem is responsible for a failure. If replacing the embeddings improves performance, the change is likely beneficial.

Hyperparameter Effects

Key empirical findings:

- **Model choice** has the largest single effect; GloVe generally outperforms skip-gram on analogy tasks at the same dimension and corpus.
- **Corpus size**: larger corpora consistently improve all models.
- **Dimension**: performance improves up to $d \approx 300$. Too small causes high bias; skip-gram and GloVe are surprisingly robust to very large d.
- **Window size**: $\sim$8 words works well for GloVe. Smaller windows capture syntactic structure; larger windows capture semantic and topical structure.

Relaxing orthogonality increases capacity. To see the effect concretely, consider an embedding space of dimension $n = 1000$. If we require vectors to be exactly orthogonal, then at most 1000 independent directions can exist. Now relax this constraint slightly and allow vectors to be nearly orthogonal, meaning their pairwise inner products satisfy

$$|\langle v_i, v_j \rangle| \leq \varepsilon.$$

High-dimensional geometry shows that the number of such vectors scales roughly as

$$M \approx 2^{\Theta(\varepsilon^2 n)}.$$

For example, if we allow a moderate deviation from orthogonality corresponding to angles of about 85° (so $\varepsilon \approx 0.087$), then $\varepsilon^2 n \approx 7.5$. This implies that the number of nearly independent directions can be on the order of

$$M \sim 2^{7.5} \approx 180$$

times larger than the strictly orthogonal limit. Instead of only 1000 independent directions, the space may support on the order of hundreds of thousands of approximately independent ones. As the dimension increases further, this growth becomes extremely rapid.

The implication for word vectors is important: embedding spaces do not need one dimension per concept. By tolerating small overlap between directions, a moderate-dimensional space can represent vastly more words or semantic features than its raw dimensionality would suggest, while still keeping them geometrically distinguishable.

From Static Vectors to Contextual Representations

Our original motivation came from sequence problems such as machine translation, where the meaning of a word depends strongly on the sentence in which it appears. This led us to develop word representations such as Word2Vec and GloVe, which replace sparse one-hot vectors with dense embeddings that capture semantic similarity.

However, these representations have an important limitation: each word is assigned a **single fixed vector**, independent of context. For example, the word "bank" receives the same embedding in both "river bank" and "investment bank," even though the intended meanings are clearly different.

This highlights the next challenge. Word embeddings provide a useful geometric representation of vocabulary, but they do not model how meaning changes as a sentence unfolds. In natural language, meaning is inherently contextual: a word cannot always be interpreted in isolation.

What we ultimately need is a **contextual representation** — a vector whose value depends not only on the word itself but also on the words surrounding it. In other words, instead of assigning each token a fixed meaning, we want a model that reads a sequence and continually updates its internal representation as new words arrive.

Word embeddings solve the representation problem; the next step is to build a model that can process sequences and produce representations that evolve with context. This is the problem

we now turn to.

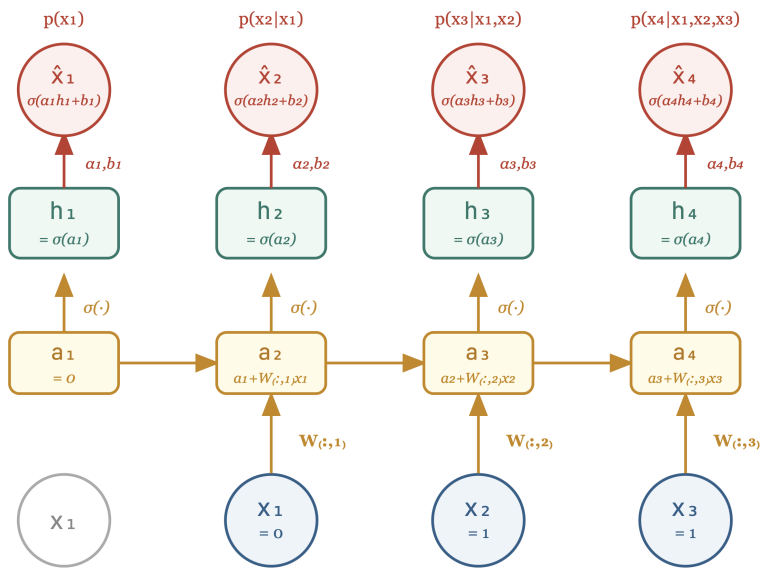

Figure 1.7: NADE architecture with pre-activation accumulation

Bridging NADE to RNN. NADE already contains the core idea that later leads to recurrent models. Instead of recomputing a new network for every conditional distribution, it incrementally updates a running quantity,

$$a_t = a_{t-1} + W_{:,t-1} x_{t-1},$$

which accumulates information about previously observed variables. Each conditional distribution is produced from this accumulated pre-activation.

From one perspective, NADE behaves as if each variable had its own neural network for conditional modeling, but parameter sharing allows these networks to be implemented efficiently through incremental updates. The accumulated vector therefore acts as a primitive form of memory.

However, the accumulation rule in NADE is limited in two ways. First, the update is purely additive and linear: the model learns *what* to add at each step via the weight columns $W_{:,t-1}$, but not *how* the accumulated state itself should evolve in response to its current value. Second, the weight columns are position-specific, so the update rule is not shared across steps; the model learns a separate transformation for each position in the sequence.

Recurrent neural networks address both limitations through one conceptual change: the state update becomes a learned function shared across all positions. Instead of a fixed additive update with position-specific weights, we introduce a learned state transition,

$$h_t = f(h_{t-1}, x_t),$$

where the same function f is applied at every step. The hidden state h_t is a fixed-dimensional representation summarizing everything observed so far.

In this view, RNNs can be understood as replacing NADE's handcrafted, position-dependent accumulation rule with a learnable, nonlinear, position-independent update that defines how information is carried forward.

1.6.3 RNNs: The Hidden State as Context

Recurrent Neural Networks (RNNs) were introduced to overcome a core limitation of feed-forward models: the inability to naturally represent sequential structure.

Many forms of data are inherently ordered. Language unfolds token by token, audio evolves continuously over time, and even images may be interpreted as ordered sequences of pixels or patches. In such settings, the prediction at time t should depend not only on the current input x_t but also on the history $x_{<t}$.

A standard neural network treats inputs independently and therefore has no mechanism for remembering past observations. RNNs address this by introducing a *hidden state*, a vector that is updated at each time step and acts as a compressed representation of everything seen so far. In this sense, the hidden state provides the contextual summary needed for sequential prediction.

Definition 1.11: Recurrent Neural Network (RNN)

A **Recurrent Neural Network** processes a sequence one element at a time, updating a hidden state h_t according to

$$h_t = \phi_1(W_{hh}h_{t-1} + W_{xh}x_t + b_h),$$

and producing an output

$$\hat{y}_t = \phi_2(W_{hy}h_t + b_y),$$

which is typically interpreted as the parameterization of a conditional distribution

$$p(y_t \mid h_t).$$

The parameters W_{hh}, W_{xh}, W_{hy} and biases b_h, b_y are shared across all time steps, allowing sequences of arbitrary length to be processed with a constant number of parameters.

The key idea is simple: rather than explicitly storing all previous inputs, the model learns a function that compresses past information into a fixed-dimensional state. This compact representation enables flexible sequence modeling — but it also introduces a fundamental optimization challenge.

Gradient Flow Through Time

Training an RNN requires learning how information should persist across many steps. This immediately raises a question: *how does learning signal propagate backward through a long sequence?*

Because each hidden state depends on the previous one, gradients must travel through the

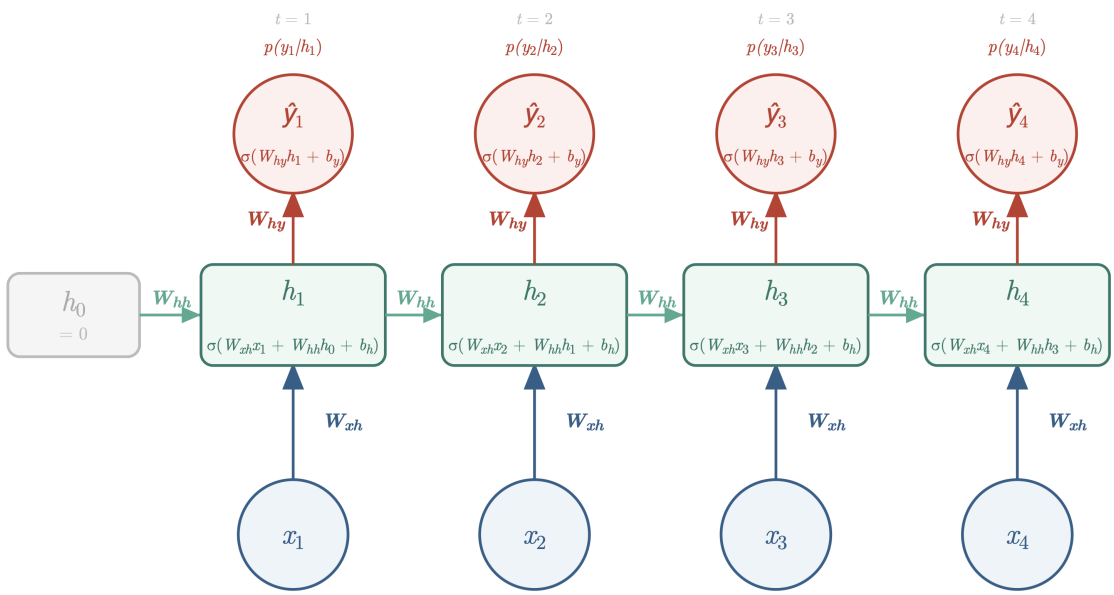

Figure 1.8: Recurrent neural network. The same transformation is reused at every time step.

entire temporal chain. The behavior of this backward flow determines whether the model can learn long-range dependencies.

Assume a sequence of length T, with a loss defined at each step:

$$L_{\text{sequence}} = \sum_{t=1}^{T} L_t.$$

To study gradient propagation, it is sufficient to examine the contribution of a single future loss term L_k $(k \geq t)$ to an earlier hidden state h_t. The full gradient is obtained by summing such contributions.

Our goal is therefore to understand how

$$\frac{\partial L_k}{\partial h_t}$$

changes as the temporal distance $(k - t)$ grows.

Backpropagation Through Time Since hidden states form a temporal chain

$$h_t \to h_{t+1} \to \cdots \to h_k,$$

the chain rule yields

$$\frac{\partial L_k}{\partial h_t} = \frac{\partial L_k}{\partial h_k} \prod_{j=t+1}^{k} \frac{\partial h_j}{\partial h_{j-1}}.$$

Thus, gradients are obtained by repeatedly multiplying Jacobians — one for each recurrent step. The central question becomes: what happens when this multiplication is repeated many times?

Jacobian of a Recurrent Step From the update equation,

$$h_j = \phi_1(W_{hh}h_{j-1} + W_{xh}x_j + b_h),$$

the Jacobian with respect to the previous hidden state is

$$\frac{\partial h_j}{\partial h_{j-1}} = D_j W_{hh},$$

where

$$D_j = \operatorname{diag}\big(\phi_1'(a_j)\big), \qquad a_j = W_{hh}h_{j-1} + W_{xh}x_j + b_h.$$

Substituting,

$$\frac{\partial L_k}{\partial h_t} = \frac{\partial L_k}{\partial h_k} \prod_{j=t+1}^{k} D_j W_{hh}.$$

The same transformation is therefore applied repeatedly as gradients move backward through time. This repeated reuse of the recurrent dynamics is the source of both the power and the instability of RNN training.

A Linearized View: Why Gradients Grow or Shrink To build intuition, we first isolate the effect of recurrence itself. Suppose the activation derivatives vary slowly across time, so that locally the network behaves approximately linearly. In this regime, we may treat

$$D_j \approx I,$$

not as an exact assumption, but as a simplifying lens that reveals the dominant dynamics of recurrence.

Under this approximation,

$$\frac{\partial L_k}{\partial h_t} \approx \frac{\partial L_k}{\partial h_k} W_{hh}^{(k-t)}.$$

The problem reduces to repeated powers of the recurrent matrix. If W_{hh} is diagonalizable,

$$W_{hh} = Q\Lambda Q^{-1}, \qquad \Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n),$$

then each component aligned with eigenvector i scales as

$$\lambda_i^{(k-t)}.$$

This immediately reveals the core phenomenon:

- If $|\lambda_i| < 1$, gradients decay exponentially.
- If $|\lambda_i| > 1$, gradients grow exponentially.

Stable propagation requires eigenvalues whose magnitudes remain close to 1, a condition that is difficult to maintain during training.

Returning to the Nonlinear Case The linearized picture provides intuition, but the full network also includes activation derivatives. Taking norms and using submultiplicativity,

$$\left\| \frac{\partial L_k}{\partial h_t} \right\| \leq \left\| \frac{\partial L_k}{\partial h_k} \right\| \prod_{j=t+1}^{k} \|D_j\| \, \|W_{hh}\|.$$

For common activations such as sigmoid or tanh,

$$|\phi_1'(x)| \leq \alpha < 1,$$

which implies $\|D_j\| \leq \alpha$. Activation derivatives therefore introduce an additional contractive effect, reinforcing the tendency for gradients to shrink over long horizons.

Combined with the recurrent dynamics, this leads to two characteristic pathologies:

- **Vanishing gradients:** early time steps receive almost no learning signal, making long-term dependencies difficult to learn.
- **Exploding gradients:** gradient norms grow uncontrollably, producing unstable parameter updates.

Why Exploding Gradients Are Easier to Handle In practice, exploding gradients are more manageable than vanishing ones. Explosions manifest as sudden large norm spikes, making them detectable during training. A simple and widely used remedy is *gradient clipping*, which rescales gradients whenever they exceed a threshold τ:

$$g \leftarrow g \cdot \frac{\tau}{\|g\|} \qquad \text{if } \|g\| > \tau.$$

Vanishing gradients, by contrast, are silent: the learning signal simply disappears, leaving the model unable to adjust parameters that influence distant past states.

> Backpropagation through time repeatedly applies the same recurrent transformation. Small deviations from unit magnitude accumulate exponentially: some directions shrink, others grow. As sequence length increases, gradients either vanish or explode, making long-range learning unstable in vanilla RNNs.

The limitation of vanilla RNNs is not representational power but optimization.

A recurrent network can, in principle, model complex sequential dependencies using a fixed set of shared parameters. In practice, however, learning is constrained by unstable gradient flow and by the need to compress an ever-growing history into a finite-dimensional hidden state. These two issues — fragile optimization and memory compression — motivate the architectural developments that follow.

1.6.4 LSTM and GRU: Fixing the Memory

The core issue is that memory is updated multiplicatively at every step, so gradients must pass through repeated products. Long-range dependencies — the kind that matter most in language — are exactly what vanilla RNNs cannot learn. The LSTM was designed specifically to fix this.

Definition 1.12: Long Short-Term Memory (LSTM)

The **LSTM** introduces a **cell state** c_t alongside h_t, controlled by three learned gates:

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \text{(forget gate: what to erase from } c_t\text{)} \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{(input gate: what new info to write)} \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{(output gate: what to expose as } h_t\text{)} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t] + b_c) \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

The cell state c_t is updated by **addition** rather than multiplication. Gradients flowing through c_t do not shrink exponentially, allowing the LSTM to learn dependencies spanning hundreds of steps. Both h_t and c_t are deterministic.

The **Gated Recurrent Unit (GRU)** is a streamlined variant that merges c_t and h_t into a single state with two gates instead of three. Computationally cheaper, and in practice often matches LSTM performance.

LSTMs and GRUs dominated sequence modeling from their introduction through the mid-2010s, producing strong results in machine translation, speech recognition, and language modeling. Their fundamental limitation remains: processing position t requires the output of position $t-1$. Training is inherently sequential and cannot be parallelized across the time dimension.

Linear accumulation revisited. The LSTM brings back an idea that first appeared in NADE: a linear additive path for carrying information forward. RNNs replaced linear accumulation with nonlinear state updates to gain expressive power, but this came at a cost: gradients shrink as they pass through many transformations, making long-range dependencies hard to learn. The LSTM addresses this by separating two roles that vanilla RNNs conflate. The hidden state handles nonlinear computation at each step, while the cell state follows a controlled additive update, gated by what the model has learned to remember or forget, giving gradients a stable path to flow backward across many steps. The key difference from NADE is that the linearity is not fixed: the forget and input gates decide at each step how much of the additive path to use. Uncontrolled linearity limits what a model can represent; controlled linearity, combined with nonlinear computation elsewhere, is what makes long-range memory possible.

Bidirectional RNNs

A standard RNN reads left to right. At position t, the hidden state h_t summarizes everything seen so far — $x_1, \ldots, x_t$ — and nothing after it. For generation this is unavoidable: you cannot condition on tokens you have not yet produced. But when the full input sequence is already available, discarding the right context is a waste.

Consider labeling each word in a sentence with its part of speech. The word "bank" is ambiguous in isolation, but not in "the bank of the river" versus "the bank approved the loan." A left-to-right RNN processing "the bank" has not yet seen the disambiguating context. A

model that could see both directions simultaneously would have no such problem.
A **bidirectional RNN** runs two RNNs over the same sequence:

$$\overrightarrow{h}_t = \mathrm{RNN}\Big(x_t,\ \overrightarrow{h}_{t-1}\Big) \quad \text{(left to right)} \tag{1.1}$$

$$\overleftarrow{h}_t = \mathrm{RNN}\Big(x_t,\ \overleftarrow{h}_{t+1}\Big) \quad \text{(right to left)} \tag{1.2}$$

The two hidden states are concatenated at each position:

$$h_t = \Big[\ \overrightarrow{h}_t\ ;\ \overleftarrow{h}_t\ \Big] \tag{1.3}$$

Now h_t encodes the full sequence context around position t — everything to its left and everything to its right. This makes bidirectional RNNs the natural choice whenever the task is to *encode* an input rather than *generate* an output. We will see this distinction become critical in the encoder-decoder architecture below.

Stacked RNNs and LSTMs

A single RNN layer maps an input sequence to a sequence of hidden states. Those hidden states are themselves a sequence — so there is nothing stopping us from feeding them into another RNN.

$$h_t^{(1)} = \mathrm{RNN}^{(1)}\Big(x_t,\ h_{t-1}^{(1)}\Big) \tag{1.4}$$

$$h_t^{(2)} = \mathrm{RNN}^{(2)}\Big(h_t^{(1)},\ h_{t-1}^{(2)}\Big) \tag{1.5}$$

$$\vdots$$

$$h_t^{(L)} = \mathrm{RNN}^{(L)}\Big(h_t^{(L-1)},\ h_{t-1}^{(L)}\Big) \tag{1.6}$$

Lower layers capture local, surface-level patterns; upper layers build more abstract representations over longer spans. The final output at each timestep is $h_t^{(L)}$, the top-layer hidden state. The same principle applies directly to LSTMs — each layer receives the hidden states of the layer below as its input sequence, and maintains its own cell state independently.

Sequence-to-Sequence Learning: The Encoder-Decoder Architecture

Every model so far models a single sequence: $p(x_1, \ldots, x_n)$. But many tasks involve two sequences — an input and an output that are related but distinct. Machine translation is the clearest example: given a French sentence of length n, produce an English sentence of length m, where $n \neq m$ in general and neither is known in advance.
What we need is a conditional model:

$$p(y_1, \ldots, y_m \mid x_1, \ldots, x_n) \tag{1.7}$$

The natural way to build this is to split the job in two. One network reads the input and builds a representation of it. Another network generates the output, conditioned on that representation. This is the **encoder-decoder** architecture.

Encoder. The encoder is a bidirectional stacked LSTM that reads the full source sequence $x_{1:n}$ from left to right and right to left. After processing the final token, we take the last hidden state and use it as a single fixed-size summary of the entire input:

$$h_t = \mathrm{LSTM}_{\mathrm{enc}}(x_t,\, h_{t-1})\,, \qquad c = h_n \tag{1.8}$$

The vector c is called the **context vector**. It is the encoder's answer to the question: *what does the decoder need to know about the source?*

Decoder. The decoder is an autoregressive LSTM initialized with c as its starting hidden state. It then generates the output sequence one token at a time, each step conditioned on everything generated so far:

$$h_t^{\mathrm{dec}} = \mathrm{LSTM}_{\mathrm{dec}}\Big(y_{t-1},\, h_{t-1}^{\mathrm{dec}}\Big)\,, \qquad h_0^{\mathrm{dec}} = c \tag{1.9}$$

$$p(y_t \mid y_{<t},\, x_{1:n}) = \mathrm{softmax}\Big(W h_t^{\mathrm{dec}} + b\Big) \tag{1.10}$$

The full model factorizes as:

$$p(y_{1:m} \mid x_{1:n}) = \prod_{t=1}^{m} p(y_t \mid y_{<t},\, x_{1:n}) \tag{1.11}$$

Training maximizes this likelihood end-to-end. Gradients flow from the decoder's predictions all the way back through c and into the encoder — the two halves are trained jointly even though they are separate networks.

The bottleneck. There is a quiet problem in this architecture. The context vector c is fixed-size. It does not matter whether the source sentence has 5 words or 50 — the encoder must compress everything into the same vector. For short sentences, this works well. For long sentences, it does not: information gets squeezed out, and translation quality degrades visibly with source length.

The problem is not the LSTM — a more powerful encoder will not fix it. The problem is structural. A single vector is a hard bottleneck, and no encoder, however deep, can overcome a fixed-size information channel when the input is arbitrarily long.

The fix is to stop forcing the decoder to read from a single summary. Instead, let the decoder look back at *all* the encoder hidden states $h_1, \ldots, h_n$ and decide, at each generation step, which parts of the source to focus on. That mechanism is **attention**, and it is the subject of the next section.

1.6.5 Attention

Dynamic Retrieval: Let the Decoder Look Back

The root cause of the bottleneck is that the decoder is cut off from the source. The fix is simple in principle: keep all the encoder hidden states available, and let the decoder look at them directly.

Let the encoder produce a sequence of hidden states

$$(h_1, h_2, \ldots, h_{T_x}),$$

where each h_j is a vector representing the j-th English word in the context of its neighbours. These are not discarded after encoding — they are kept, and the decoder can access all of them at every step.

At each decoding step, rather than reading from a fixed summary, the decoder computes a *dynamic* context vector as a weighted combination:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij}\, h_j.$$

The weights α_{ij} reflect how relevant each English position is to the current German word being generated. They are computed by scoring the compatibility between the current decoder state s_{i-1} and each encoder state h_j, then normalising via softmax:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}.$$

The weights form a probability distribution over source positions. When generating the German word *schläft* (sleeps), the decoder places high weight on the English word *sleeps* and low weight on everything else. When generating a noun, it shifts attention to the relevant English noun. The retrieval is dynamic and input-dependent — it changes at every step based on what is being generated. This mechanism is *attention*.

The bottleneck is gone. The decoder no longer has to extract everything from one fixed vector. It looks back at the full English sequence and selectively retrieves what it needs, when it needs it.

How should compatibility be scored? Everything hinges on computing e_{ij} well. This is the design question that drives the rest of the section.

First Design: Score with a Network

The most expressive option is to learn a small feed-forward network that takes the decoder state s and encoder state h and outputs a score:

$$e = v^\top \tanh(W_1 s + W_2 h).$$

The network learns what compatibility means directly from translation data. It can capture complex, nonlinear interactions between the decoder and encoder states.

This works well. But it has a structural cost. A feed-forward network must be evaluated for *every pair* of decoder and encoder positions — and decoder steps are sequential, each depending on the previous hidden state, so none of this can be parallelised. On long sentences, this is slow.

Second Design: Score with a Dot Product

A simpler alternative: replace the network with a dot product,

$$e = s^\top h.$$

No learned parameters, no nonlinearity. Much cheaper to compute, and competitive in practice.

But there are two problems with taking dot products between raw hidden states.

The relevance problem. A dot product measures *raw similarity* between vectors. The German decoder state and an English encoder state will score highly if their representations happen to be close — regardless of whether they are actually relevant to each other in this sentence. Similarity is not relevance.

The scale problem. When the hidden state dimension d is large, dot products grow in magnitude proportionally to d. This pushes the softmax into saturation, where gradients become very small and training stalls.

Both problems share the same root: we are comparing vectors in the wrong space.

The Fix: Project Before You Compare

The solution is to stop comparing raw hidden states directly. Instead, first project each state into a smaller, learned interaction space, then compare there.

Consider a concrete example. The English source sentence is:

"The black cat sleeps on the rug."

The German decoder has produced *"Die schwarze Katze"* and is now generating the next word. It needs to find which English word to draw from next — in this case, *sleeps*. The decoder state s encodes everything the decoder knows at this moment: what has been generated so far, what grammatical context is expected next. But s lives in the German decoder's representation space, and the English encoder states $h_1, \ldots, h_n$ live in the encoder's representation space. They were produced by different networks, for different purposes. Comparing them directly is like asking whether two rulers agree when one measures in centimetres and the other in inches — the numbers are not comparable without a conversion.

The fix is to learn a projection for each role. The German decoder state is *asking a question*: which English word is relevant to the German word I am about to generate? Each English encoder state is *advertising what it contains*: here is the kind of information I represent. Learn a projection W_Q that maps the German decoder state into a **query** — a vector encoding what is being looked for:

$$q = s\, W_Q.$$

Learn a separate projection W_K that maps each English encoder state into a **key** — a vector encoding what that word is advertising it contains:

$$k_j = h_j\, W_K.$$

Both projections map into the same d_k-dimensional space, where comparison is now meaningful. The compatibility score between the German decoder and each English source word becomes:

$$e_j = \frac{q^\top k_j}{\sqrt{d_k}},$$

where the $\sqrt{d_k}$ factor corrects for the scale problem directly.

In the example above, when the German decoder is about to generate *schläft*, its query should score highly against the key of *sleeps* and low against *cat*, *black*, or *rug*. The model is not told this explicitly — it learns it because that alignment is what produces correct translations.

The scale problem is fixed: projecting into a d_k-dimensional space keeps dot products well-behaved regardless of the original dimension.

The relevance problem is fixed: W_Q and W_K are learned from translation data. The German decoder state learns to produce a query that aligns with the key of the English word it actually needs, not just the English word with the most similar raw representation. This emerges from training, not from explicit design.

The roles are separated: the German decoder state projects into a query — what it is looking for. The English encoder state projects into a key — what it is offering. These are independent projections learned for different purposes.

The Query/Key Intuition in Translation

The German decoder is generating the next word and sends out a **query**: a vector encoding what kind of English content it needs right now. Each English word holds up a **key**: a vector encoding what kind of information it contains. The dot product between the query and each key measures how well they match in the learned interaction space.

Returning to the example: when generating *schläft*, the decoder query aligns strongly with the key of *sleeps* and weakly with *cat* or *black*. When generating *Teppich* (rug), the same mechanism shifts attention entirely to *rug*.

- **Query** = what the German decoder needs at this step.
- **Key** = what each English word is advertising it contains.
- **Match** = dot product in the shared projection space.

The projections W_Q and W_K learn to map decoder and encoder states into a common space where this matching reflects contextual relevance, not raw representational similarity. The same English word *sleeps* will produce a different effective key depending on what surrounds it — the projection space is learned globally, but the representations entering it carry context from the recurrent encoder.

> The query/key framing is a useful label, not a design constraint. The projections do not explicitly encode "questions" or "answers." They are learned linear maps, shaped entirely by what produces the best translations. The terminology helps intuition; the math does the work.

What Gets Retrieved: The Value Projection

Once the attention weights are computed, the model takes a weighted sum over the encoder states. But a weighted sum of *what*?

Using the key vectors k_j would conflate two things that should be independent: how a word is *found* and what it *contributes* once found. A key is tuned to be discoverable — it is optimised for matching queries. But what the German decoder actually needs once it has located the right English word is its meaning: its semantic content, its grammatical role, its relationship to surrounding words. These are different things and should not be tied to the same vector.

The fix is a third projection W_V, producing a **value** vector for each English encoder state:

$$v_j = h_j W_V.$$

Queries and keys determine *how much* each English word contributes. Values determine *what* is actually contributed. The output at each decoding step is:

$$\text{output} = \sum_j \alpha_j v_j.$$

Returning to the example: *sleeps* is found because its key matches the German decoder's query for a verb. But what gets retrieved is its value — the rich semantic and contextual representation of *sleeps* built by the English encoder, not merely the signal that made it findable. A word can be easy to locate for one reason and contribute something entirely different. This separation is the point.

The Q/K/V structure is not an arbitrary convention. It is the result of fixing three concrete problems in sequence: scale, relevance, and the separation of lookup from contribution.

Definition 1.13: Scaled Dot-Product Attention

Given input representations, compute three projections for each position:

$$q = xW_Q, \qquad k = xW_K, \qquad v = xW_V.$$

For queries Q, keys K, and values V stacked into matrices:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- **Query** (Q): what each position is looking for.
- **Key** (K): what each position is advertising it contains.
- **Value** (V): what each position contributes once found.
- $\sqrt{d_k}$: scaling factor that prevents dot products from growing large and saturating the softmax.

The output at each position is a weighted sum of values, where the weights are determined entirely by query–key compatibility.

Multiple Heads: Multiple Notions of Relevance

A single set of projections W_Q, W_K, W_V learns one notion of relevance. At each decoding step, the attention weights form a single distribution over the source sequence — one joint decision about what to retrieve. The problem is that relevance is not one thing.

Even within a single sentence, generating one word can require answering several structurally different questions simultaneously. When the German decoder generates *schläft*, it needs to know: *what is the verb?* (to find *sleeps*), *what is the subject?* (to enforce grammatical agreement with *cat*), and *what is the object?* (to understand the full predicate). These are not the same question. A single attention distribution must place its weight somewhere — it cannot simultaneously peak at *sleeps*, *cat*, and *rug* without diluting each. The model is forced to average over relationships that should be kept separate.

The solution is to run H independent attention heads in parallel, each with its own projections $W_Q^{(h)}$, $W_K^{(h)}$, $W_V^{(h)}$:

$$\text{head}_h = \text{Attention}(QW_Q^{(h)},\, KW_K^{(h)},\, VW_V^{(h)}).$$

Each head operates in its own subspace and learns its own notion of what makes two positions relevant to each other. One head may learn to track syntactic agreement, another lexical correspondence, another long-range coreference. Because they run in parallel on the same input, the model simultaneously captures all the relationships a single head would be forced to trade off. Every head runs on every input, every time — there is no routing and no competition. The specialisation emerges entirely from training.

The outputs of all heads are concatenated and passed through a final projection W^O:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, W^O.$$

The final projection does not merely stack the outputs — it learns to mix information across heads, so that what one head noticed about syntax and another noticed about semantics can be combined into a single representation that reflects both.

Definition 1.14: Multi-Head Attention (MHA)

Run H independent attention heads in parallel, each with its own learned projections:

$$\text{head}_h = \text{Attention}(QW_Q^{(h)},\, KW_K^{(h)},\, VW_V^{(h)}).$$

Concatenate all head outputs and project back to the model dimension:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, W^O.$$

- Each head operates in a subspace of dimension $d_k = d_{\text{model}}/H$.
- Each head learns an independent notion of relevance.
- All heads run on the same input simultaneously — there is no routing, no competition. Specialisation emerges from training.
- W^O learns to mix information across heads into a single unified representation.

Self-Attention: Turning the Mechanism Inward

The mechanism developed so far is *cross-attention*: the German decoder attends to the English encoder. The query comes from the German side; the keys and values come from the English side. The two sequences are different.

But now consider the English sentence itself, before translation begins. The meaning of each English word depends on the words around it. The word *bank* means something different beside *river* than beside *loan*. A recurrent encoder captures this by passing information forward step by step, but this forces early words to carry their influence through a long chain of hidden states to reach distant ones — a slow, sequential, and lossy process.

The same attention mechanism can be applied within a single sequence. Each English word produces its own query, key, and value, and attends over all other words in the same sentence. This is *self-attention*: the sequence attending to itself.

The effect is significant. Any two positions interact directly in a single operation — *bank* can immediately attend to *river* or *loan* to resolve its meaning, without the relationship having to propagate through every intermediate hidden state. Long-range dependencies are no harder to capture than short-range ones, and the entire computation runs in parallel.

One property that recurrence provided for free must now be imposed explicitly: causal order. When generating German word by word, the prediction at position t must depend only on earlier positions. Recurrence guaranteed this structurally. Self-attention has no such constraint by default — every position can attend to every other, including future ones. The fix is to mask future positions before the softmax, forcing their weights to zero. This is *masked self-attention*.

Definition 1.15: Cross-Attention

The queries come from one sequence (the decoder); the keys and values come from a different sequence (the encoder):

$$\text{CrossAttention}(S_{\text{dec}},\, S_{\text{enc}}) = \text{Attention}(S_{\text{dec}}W_Q,\; S_{\text{enc}}W_K,\; S_{\text{enc}}W_V).$$

Each decoder position asks: *which encoder position is most relevant to what I am generating right now?* In English-to-German translation, the German decoder queries the English encoder at every generation step.

Definition 1.16: Self-Attention

The queries, keys, and values all come from the *same* sequence:

$$\text{SelfAttention}(X) = \text{Attention}(XW_Q,\; XW_K,\; XW_V).$$

Each position attends over all other positions in the same sequence, allowing direct interaction between any two tokens regardless of their distance. In **masked self-attention**, future positions are set to $-\infty$ before the softmax so the model cannot

attend to tokens it has not yet generated:

$$e_{ij} = -\infty \quad \text{for all } j > i.$$

Recurrence was never the goal. Looking back at the full arc: recurrence was not a fundamental principle. It was a mechanism for propagating dependencies step by step because no direct alternative existed. Attention is that alternative — any two positions interact directly, without a sequential chain between them.

Cross-attention lets the German decoder retrieve from the English encoder. Self-attention lets each sequence understand itself. Both are the same mechanism: compute queries, compute keys and values, weight by compatibility, aggregate. The only difference is where the queries and keys come from.

The pieces are now in place. The Transformer is the architecture that builds entirely around this idea, replacing recurrence with attention throughout, and surrounding it with the components needed to make it work at scale.

1.6.6 Transformer

Formal definition. The input X is first normalized:

$$\tilde{X} = \text{LayerNorm}(X).$$

Normalization stabilizes training and keeps representations well scaled before projections are applied.

For each head h, three projections are computed:

$$Q_h = \tilde{X} W_Q^{(h)}, \qquad K_h = \tilde{X} W_K^{(h)}, \qquad V_h = \tilde{X} W_V^{(h)}.$$

The attention output for head h is:

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_h^\top + M}{\sqrt{d_k}}\right) V_h,$$

where M is the causal mask:

$$M_{ij} = \begin{cases} 0 & j \le i, \\ -\infty & j > i. \end{cases}$$

Setting future positions to $-\infty$ before the softmax maps their weights to zero, ensuring that position i attends only to tokens $(x_1, \ldots, x_i)$ and enforcing autoregressive generation.

All heads are concatenated and projected:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, W^O \in \mathbb{R}^{N \times d_{\text{model}}},$$

where $d_{\text{model}} = H \cdot d_k$.

The result is added back via a residual connection:

$$X_{\text{att}} = X + \text{MHA}(X).$$

Residual connections preserve information across layers and give gradients a direct path through the network, stabilizing optimization.

From Text to Input Vectors

Given a tokenized sequence

$$(x_1, x_2, \ldots, x_N),$$

each token is mapped to an embedding vector using a learned embedding matrix

$$E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}.$$

This produces

$$X \in \mathbb{R}^{N \times d_{\text{model}}}.$$

Unlike recurrent or convolutional models, the Transformer has no built-in notion of sequence order. Self-attention treats tokens as a set, so information about position must be injected explicitly. For this reason, **positional encodings** are added to the token embeddings.

These positional vectors have the same dimensionality as the embeddings, allowing them to be summed:

$$X = X^{(0)} = \text{TokenEmbedding} + \text{PositionEmbedding}.$$

In the original Transformer paper, sinusoidal positional encodings were proposed, where each dimension follows a sine or cosine function of a different frequency. This design allows the model to represent relative positions through simple linear operations. Later models often use learned positional embeddings instead; empirically, both approaches work well in practice.

The resulting matrix X combines both token identity and positional information, and becomes the input to the first decoder layer.

Masked Self-Attention

Self-attention allows every token to attend to other tokens in the sequence. In decoder-only models, attention is **masked** so that a token cannot see future tokens.

For head h:

$$Q_h = XW_Q^h, \quad K_h = XW_K^h, \quad V_h = XW_V^h.$$

Attention weights are computed as

$$\text{Attention}(Q_h, K_h, V_h) = \text{softmax}\left(\frac{Q_h K_h^\top + M}{\sqrt{d_k}}\right) V_h,$$

where M is a causal mask:

$$M_{ij} = \begin{cases} 0 & j \leq i, \\ -\infty & j > i. \end{cases}$$

The mask ensures that position i only attends to tokens

$$(x_1, \ldots, x_i),$$

which enforces autoregressive generation.

Each head outputs

$$(N, d_{\text{head}}).$$

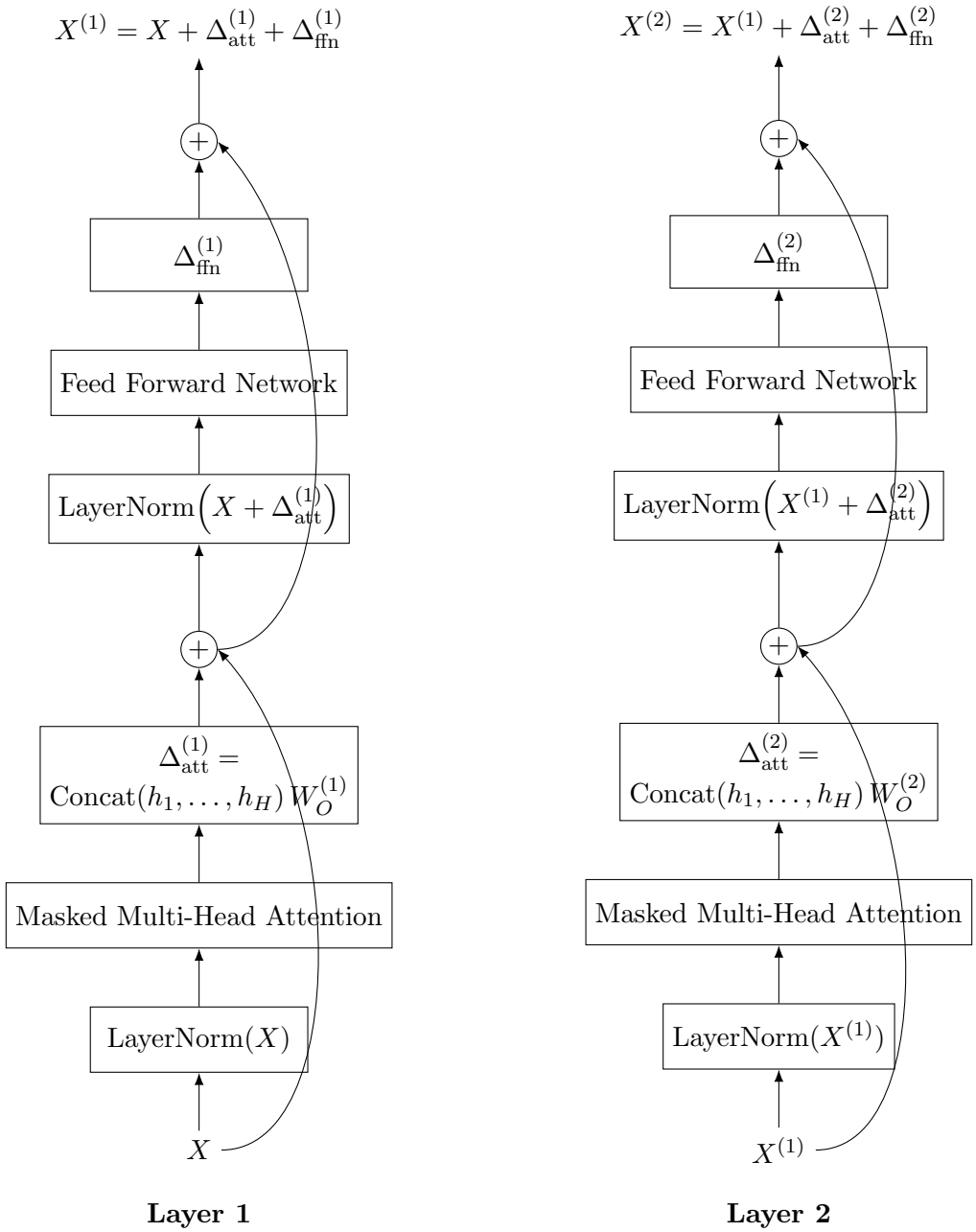

Figure 1.9: Pre-Norm Decoder-Only Transformer with Depth-Indexed Residual Updates

All heads are concatenated:

$$\text{Concat}(\text{head}_1, \ldots, \text{head}_H) \in \mathbb{R}^{N \times (H d_{\text{head}})}.$$

Since

$$d_{\text{model}} = H d_{\text{head}},$$

the output returns to shape

$$(N, d_{\text{model}})$$

after a linear projection.

Interpretation. Attention is a global communication step in which tokens exchange contextual information.

Intuition: What Happens Inside Multi-Head Attention

When the input representation X enters multi-head attention, it is sent to all attention heads in parallel. Each head receives the same input but uses its own learned projection matrices, allowing different heads to focus on different types of relationships.

Inside a single head, each token is transformed into three vectors: a query, a key, and a value. The query represents what the token is looking for, the key represents what information the token offers, and the value represents the information that will actually be passed to other tokens.

A useful intuition is to think in terms of questions and answers. Consider the sentence

"A red sports car sped past."

One head might focus on identifying colors. The token *car* produces a query that effectively asks, "What color am I?" The token *red* produces a key that matches this query, leading to a high attention score.

Another head may focus on the type of vehicle. In this case the token *car* asks, "What kind of car am I?" The token *sports* provides a key that matches this query.

A different head may focus on actions or motion. Here the token *car* might ask, "How am I moving?" The token *sped* produces a key that matches this query.

> The roles described above are only an intuition. Attention heads are not designed to detect specific semantic properties; they simply learn useful patterns of interaction between tokens during training.

These scores are normalized with a softmax to produce attention weights. The weights do not transfer information by themselves; instead, they determine how strongly the value vectors of different tokens are combined. The output of attention is therefore a weighted mixture of values, meaning that each token gathers contextual information from the tokens it attends to.

In decoder, attention is causally masked. Before softmax, scores for future positions are set to $-\infty$, ensuring that a token can attend only to itself and earlier tokens. This enforces autoregressive generation, where prediction at position i depends only on

$$(x_1, \ldots, x_i).$$

Before entering attention, the input is normalized using LayerNorm. Normalization stabilizes training and keeps representations well scaled.

Each head performs this process independently, learning its own interaction patterns. The outputs of all heads are then concatenated and projected back to the model dimension,

producing a contextual representation where each token contains information gathered from multiple perspectives.

Structure of a Decoder Layer

Each decoder layer contains two main blocks:

1. Masked multi-head self-attention
2. Position-wise feed-forward network (FFN)

The computation in a pre-norm decoder layer is

$$\begin{aligned}
\Delta_{\text{att}}^{(l)} &= \text{MHA}\Big(\text{LayerNorm}(X^{(l)})\Big)\,, \\
X_{\text{att}}^{(l)} &= X^{(l)} + \Delta_{\text{att}}^{(l)}, \\
\Delta_{\text{ffn}}^{(l)} &= \text{FFN}\Big(\text{LayerNorm}(X_{\text{att}}^{(l)})\Big)\,, \\
X^{(l+1)} &= X_{\text{att}}^{(l)} + \Delta_{\text{ffn}}^{(l)}.
\end{aligned}$$

Residual connections preserve information and stabilize optimization.

Feed-Forward Network (Upscaling and Downscaling)

After multi-head attention, each head produces a contextual representation of the sequence. These head outputs are concatenated and projected back to the model dimension:

$$(N, d_{\text{model}}).$$

At this stage, each token representation already contains information gathered from other tokens. Attention has performed a global communication step: it decides *which tokens are relevant* and mixes their information together.

The next step is to transform this contextualized representation. This is the role of the feed-forward network (FFN).

Unlike attention, the FFN does not mix information across tokens. Instead, it is applied independently to each token vector:

$$\text{FFN}(x) = W_2(ReLU(W_1 X + b_1)) + b_2.$$

Thus, the computation inside a decoder layer follows the pattern

$$\text{multi-head attention} \;\rightarrow\; \text{concatenation and projection} \;\rightarrow\; \text{FFN}.$$

Intuitively, attention gathers information from the sequence, while the FFN processes and transforms the result locally for each token.

The FFN performs two transformations:

1. **Upscaling (Expansion):**

$$d_{\text{model}} \rightarrow d_{\text{ff}},$$

   where typically

$$d_{\text{ff}} \approx 4\, d_{\text{model}}.$$

2. **Downscaling (Projection):**

$$d_{\text{ff}} \to d_{\text{model}}.$$

For example, if

$$d_{\text{model}} = 640,$$

the FFN may perform

$$640 \to 2560 \to 640.$$

Why expansion? The attention block mainly mixes information linearly across tokens. The expansion step gives the model additional representational space to build richer nonlinear features from this contextual information. The projection step then compresses these features back to the model dimension so that the next layer can operate on the same tensor shape.

Important properties:

- One FFN per layer.
- Shared across all token positions.
- Different layers have different FFN parameters.

Intuition: What the FFN Does After Attention

At this point, each row corresponds to a single token. That row already contains contextual information coming from all attention heads and from all tokens that were allowed to attend to it.

The role of the FFN is to take this collected information and *process it*. Unlike attention, which moves information between tokens, the FFN works on one token at a time. It asks:

> Given everything this token has learned from the sequence, how should its representation be updated?

The expansion step allows the model to temporarily move into a higher-dimensional space where different signals from the attention heads can interact nonlinearly. The final projection compresses this computation back to d_{model}, producing an update to the token representation.

Because of the residual connection, the FFN output is added to the incoming representation:

$$\text{new representation} = \text{previous representation} + \text{FFN update}.$$

Thus, the FFN can be interpreted as producing a *delta* — a refinement that adjusts the token after contextual information has been gathered by attention.

In this way, attention gathers information from across the sequence, while the FFN integrates, reshapes, and incrementally refines each token representation.

Interpretation. Attention determines how tokens interact globally; the FFN transforms each token representation locally after this interaction.

Stacking Decoder Layers

Decoder layers are stacked:

$$\text{Layer}_1 \to \text{Layer}_2 \to \cdots \to \text{Layer}_N.$$

The tensor shape remains constant:

$$(N, d_{\text{model}}),$$

while the representations become increasingly contextual and abstract.

Each layer performs another round of:

- global communication (attention),
- local transformation (FFN).

Intuition: Layer-by-Layer Refinement

At a conceptual level, each decoder layer applies two incremental updates to the representation (omitting LayerNorm for clarity):

$$X \leftarrow X + \Delta_{\text{attention}}, \qquad X \leftarrow X + \Delta_{\text{FFN}}.$$

Here, multi-head attention produces a contextual update that incorporates information from other tokens, while the feed-forward network produces a local nonlinear update applied independently to each token. Because of the residual connections, these modules do not replace the representation; instead, they add to it. Across many layers, the model repeatedly transforms the initial token embeddings into progressively richer contextual representations. The final vectors produced by the Transformer are not predictions themselves. Rather, they are contextual representations designed so that a simple linear projection into vocabulary space yields accurate next-token probabilities. In practice, training encourages these vectors to align strongly (under dot-product similarity) with the embedding direction of the correct next token. From this perspective, the architecture can be viewed as repeatedly transforming token representations into forms that make prediction increasingly easy. This continual transformation of representations motivates the name **Transformer**.

Residual Refinement and Prediction

Each layer does not rewrite X; it corrects it. The update rule for layer l is

$$X^{(l)} = X^{(l-1)} + \Delta^{(l)}_{\text{attn}} + \Delta^{(l)}_{\text{ffn}}.$$

Thus, depth accumulates refinements rather than replacements. Unrolling to layer L gives

$$X^{(L)} = X^{(0)} + \sum_{l=1}^{L} \left(\Delta^{(l)}_{\text{attn}} + \Delta^{(l)}_{\text{ffn}} \right).$$

Connection to Residual Networks. This is exactly the structural form of a residual network, where each block computes

$$x_{l+1} = x_l + F_l(x_l).$$

In the Transformer, the residual function $F_l(\cdot)$ is decomposed into two structured components: attention (communication across positions) and a feed-forward network (position-wise transformation). Like ResNet, each layer contributes an additive update, never a substitution.

Prediction. After the final layer,

$$X^{(L)} \in \mathbb{R}^{N \times d_{\text{model}}}.$$

Let $x_i^{(L)} \in \mathbb{R}^{1 \times d_{\text{model}}}$ denote the i-th row of $X^{(L)}$. To obtain scores over the vocabulary, we project into vocabulary space via the weight matrix

$$W_{\text{vocab}} \in \mathbb{R}^{d_{\text{model}} \times |\mathcal{V}|}.$$

$$\ell_i = x_i^{(L)} W_{\text{vocab}}.$$

Equivalently, the score for word w is

$$s_i(w) = \langle x_i^{(L)}, e_w \rangle.$$

Since $x_i^{(L)}$ is itself a sum of residual refinements, every layer contributes additively to every vocabulary score.
Applying softmax yields the next-token distribution:

$$p(w \mid x_{<i}) = \frac{\exp(s_i(w))}{\sum_{w'} \exp(s_i(w'))}.$$

Training Objective

The decoder-only Transformer is trained as an autoregressive language model. The joint distribution over a sequence $(x_1, \ldots, x_N)$ is factorized as

$$p_\theta(x_1, \ldots, x_N) = \prod_{i=1}^{N} p_\theta(x_i \mid x_{<i}).$$

At each position i, the model produces scores

$$s_i(w) = \langle x_i^{(L)}, e_w \rangle,$$

which define a conditional distribution via softmax:

$$p_\theta(w \mid x_{<i}) = \frac{\exp(s_i(w))}{\sum_{w'} \exp(s_i(w'))}.$$

Maximum Likelihood. Training maximizes the log-likelihood of observed sequences:

$$\mathcal{L}(\theta) = \sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}).$$

Equivalently, we minimize the negative log-likelihood (cross-entropy loss):

$$\mathcal{J}(\theta) = -\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}).$$

Parallel Training. During training, the full sequence

$$(x_1, x_2, \ldots, x_N)$$

is provided to the model simultaneously. After passing through all decoder layers, the model produces

$$X^{(L)} = \begin{bmatrix} x_1^{(L)} \\ x_2^{(L)} \\ \vdots \\ x_N^{(L)} \end{bmatrix} \in \mathbb{R}^{N \times d_{\text{model}}}.$$

Because causal masking prevents position i from attending to positions $> i$, each row $x_i^{(L)}$ depends only on the prefix $x_{<i}$. Thus, $x_i^{(L)}$ is precisely the representation required to model the conditional distribution

$$p_\theta(x_i \mid x_{<i}).$$

Since this holds simultaneously for every position, the matrix $X^{(L)}$ contains all prefix-conditioned representations at once. All conditional probabilities

$$p_\theta(x_1 \mid \varnothing),\ p_\theta(x_2 \mid x_1),\ \ldots,\ p_\theta(x_N \mid x_{<N})$$

are therefore obtained from the same final representation matrix in a single forward pass—an efficiency that would be impossible without causal masking.

Inference (Autoregressive Generation). During inference, only a prefix

$$(x_1, \ldots, x_k)$$

is provided to the model. After processing this prefix, the decoder produces a representation matrix whose final row encodes the prefix-conditioned state for position k.
From this representation, the model computes the distribution

$$p_\theta(x_{k+1} \mid x_{\leq k}),$$

and a token x_{k+1} is selected (e.g., by sampling or argmax).
The generated token is then appended to the input sequence:

$$(x_1, \ldots, x_k, x_{k+1}),$$

and the process repeats. At each step, the model evaluates

$$p_\theta(x_{i+1} \mid x_{\leq i}),$$

growing the sequence one token at a time.
Unlike training, future tokens are not available, so generation must proceed sequentially.

Summary

The Transformer decoder alternates between masked self-attention and feed-forward computation. Masked attention enforces the autoregressive constraint, ensuring that each position attends only to its prefix; the FFN then expands and compresses the feature space at each position independently, increasing representational capacity. Stacking many such layers accumulates additive refinements over the residual stream, progressively sharpening token representations until a final linear projection and softmax yield next-token probabilities. This architecture simultaneously supports efficient parallel training—by processing all prefix-conditioned representations in one forward pass—and flexible autoregressive generation, properties that together underpin the scalability of modern large language models.

Worked Example: A Complete Pre-Norm Transformer Block

The preceding sections described the Transformer's components — LayerNorm, multi-head self-attention, residual connections, and the feed-forward network — at the level of equations and diagrams. This section works through a single Pre-Norm block numerically from raw token embeddings to the final output, tracking every matrix computation with concrete numbers.

The Pre-Norm block in one equation. LayerNorm is applied *before* each sub-layer, not after:

$$X_{\text{attn}} = X + \text{MHA}\big(\text{LN}(X)\big), \qquad X_{\text{out}} = X_{\text{attn}} + \text{FFN}\big(\text{LN}(X_{\text{attn}})\big).$$

The raw input X flows unchanged through both skip connections; only the normalized branch enters each sub-layer.

Toy configuration.

Symbol	Meaning	Value
n	sequence length (tokens)	3
d_{model}	embedding dimension	4
h	number of attention heads	2
d_{head}	per-head dimension ($= d_{\text{model}}/h$)	2

Step 1 — Raw input.

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 4}.$$

Rows are tokens w_1, w_2, w_3. X is the *residual stream* — the block only ever adds increments to it; it is never overwritten.

Step 2 — LayerNorm before attention. Applied independently to each row: $\text{LN}(x) = (x-\mu)/\sigma$ where μ, σ are the row mean and standard deviation.
w_1**:** $\mu = 0.5,\ \sigma = 0.5 \ \Rightarrow\ \tilde{x}_1 = [1, -1, 1, -1]$.
w_2**:** $\mu = 0.5,\ \sigma = 0.5 \ \Rightarrow\ \tilde{x}_2 = [-1, 1, 1, -1]$.
w_3**:** $\mu = 0.75,\ \sigma \approx 0.433 \ \Rightarrow\ \tilde{x}_3 \approx [0.58, 0.58, -1.73, 0.58]$.

$$\tilde{X} = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 0.58 & 0.58 & -1.73 & 0.58 \end{bmatrix}.$$

Projections use $\tilde{X}$, skip connections use X. All query/key/value matrices are computed from $\tilde{X}$. The raw X appears *only* in the residual additions below.

Step 3 — Multi-head self-attention. Each head h projects $\tilde{X}$ into 2-dimensional Q/K/V spaces via $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{4\times 2}$.
Head 1 — projections:

$$W_Q{}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K{}^{(1)} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_V{}^{(1)} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

$$Q^{(1)} = \begin{bmatrix} 2 & -2 \\ 0 & 0 \\ -1.15 & 1.15 \end{bmatrix}, \quad K^{(1)} = \begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -1.15 & -0.58 \end{bmatrix}, \quad V^{(1)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1.16 & -1.15 \end{bmatrix}.$$

Scores, causal mask ($-\infty$ for $j > i$), and softmax:

$$S^{(1)}_{\text{masked}} = \begin{bmatrix} 6 & -\infty & -\infty \\ 0 & 0 & -\infty \\ -3.45 & 1.15 & 0.66 \end{bmatrix}, \quad A^{(1)} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 0.006 & 0.61 & 0.38 \end{bmatrix}.$$

Head 2 — projections:

$$W_Q{}^{(2)} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_K{}^{(2)} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad W_V{}^{(2)} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

$$S^{(2)}_{\text{masked}} = \begin{bmatrix} 0 & -\infty & -\infty \\ -2 & 0 & -\infty \\ 1.15 & -1.15 & -1.33 \end{bmatrix}, \quad A^{(2)} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.12 & 0.88 & 0 \\ 0.84 & 0.10 & 0.06 \end{bmatrix}.$$

Two heads, two patterns. Head 1 has w_3 attending mostly to w_2 (0.61); Head 2 has w_3 attending mostly to w_1 (0.84). Each head independently extracts a different relational signal from the same sequence — the core motivation for multi-head attention.

Step 4 — Contextual outputs and concatenation. $C^{(h)} = A^{(h)}V^{(h)} \in \mathbb{R}^{3\times 2}$:

$$C^{(1)} \approx \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0.44 & -0.44 \end{bmatrix}, \quad C^{(2)} \approx \begin{bmatrix} 2.00 & -1.00 \\ 0.24 & -0.12 \\ 1.61 & -0.77 \end{bmatrix}.$$

Concatenate heads and apply output projection $W_O = I$:

$$\Delta_{\text{attn}} = [C^{(1)} \mid C^{(2)}] = \begin{bmatrix} 0 & 0 & 2.00 & -1.00 \\ 0 & 0 & 0.24 & -0.12 \\ 0.44 & -0.44 & 1.61 & -0.77 \end{bmatrix} \in \mathbb{R}^{3\times 4}.$$

Step 5 — First residual connection.

$$X_{\text{attn}} = X + \Delta_{\text{attn}} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 2.00 & -1.00 \\ 0 & 0 & 0.24 & -0.12 \\ 0.44 & -0.44 & 1.61 & -0.77 \end{bmatrix} \approx \begin{bmatrix} 1 & 0 & 3.00 & -1.00 \\ 0 & 1 & 1.24 & -0.12 \\ 1.44 & 0.56 & 1.61 & 0.23 \end{bmatrix}.$$

Step 6 — LayerNorm before FFN.

$$X_{\text{FFN-norm}} = \text{LN}(X_{\text{attn}}) \approx \begin{bmatrix} 0.39 & -0.65 & 1.43 & -1.17 \\ -0.94 & 0.89 & 1.11 & -1.05 \\ 1.68 & -0.15 & -0.69 & -0.84 \end{bmatrix}.$$

Step 7 — Feed-forward network. The FFN $\text{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2$ is applied token-wise to $X_{\text{FFN-norm}}$:

$$\Delta_{\text{FFN}} \approx \begin{bmatrix} 0.22 & -0.08 & 0.31 & -0.06 \\ -0.18 & 0.27 & 0.12 & -0.21 \\ 0.36 & -0.04 & -0.11 & 0.14 \end{bmatrix}.$$

Step 8 — Second residual connection.

$$X_{\text{out}} = X_{\text{attn}} + \Delta_{\text{FFN}} \approx \begin{bmatrix} 1.22 & -0.08 & 3.31 & -1.06 \\ -0.18 & 1.27 & 1.36 & -0.33 \\ 1.80 & 0.52 & 1.50 & 0.37 \end{bmatrix}.$$

The residual stream view. The block is three matrices added together:

$$\boxed{X_{\text{out}} = X + \Delta_{\text{attn}} + \Delta_{\text{FFN}}.}$$

$$\underbrace{\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}}_{X} + \underbrace{\begin{bmatrix} 0 & 0 & 2.00 & -1.00 \\ 0 & 0 & 0.24 & -0.12 \\ 0.44 & -0.44 & 1.61 & -0.77 \end{bmatrix}}_{\Delta_{\text{attn}}} + \underbrace{\begin{bmatrix} 0.22 & -0.08 & 0.31 & -0.06 \\ -0.18 & 0.27 & 0.12 & -0.21 \\ 0.36 & -0.04 & -0.11 & 0.14 \end{bmatrix}}_{\Delta_{\text{FFN}}} \approx X_{\text{out}}.$$

Key takeaway. A transformer block never *overwrites* representations — it *accumulates* them. Attention writes a context-aware increment Δ_{attn}; the FFN writes a local nonlinear increment Δ_{FFN}. Both gradients flow unimpeded backward through the skip connections, bypassing every nonlinearity — this is why deep transformers are trainable despite their depth.

Encoder–Decoder Transformer for Sequence Transduction

The decoder-only architecture described above is designed for language modelling: given a sequence of tokens, predict the next one. A different class of tasks requires **sequence transduction** — mapping one sequence to another, such as translating English to French. For this, the Transformer adopts an encoder–decoder structure.

The Encoder. The encoder processes the entire source sequence at once. Because the full source is available before decoding begins, there is no need to restrict tokens from attending to future positions — the causal mask is removed. Each encoder layer applies unmasked self-attention followed by a feed-forward network:

$$\begin{aligned} X' &= X + \mathrm{MHA}(\mathrm{LayerNorm}(X)), \\ Y &= X' + \mathrm{FFN}(\mathrm{LayerNorm}(X')). \end{aligned}$$

After multiple encoder layers, the source sequence is represented as a matrix of contextual vectors:

$$H^{\mathrm{enc}} \in \mathbb{R}^{N \times d_{\mathrm{model}}},$$

where each row is a richly contextualized representation of a source token.

Cross-Attention. The decoder generates the target sequence autoregressively, one token at a time. In addition to masked self-attention over previously generated tokens, each decoder layer contains a **cross-attention** block that connects the decoder to the encoder output.

In cross-attention, queries come from the decoder and keys and values come from the encoder:

$$Q = X^{\mathrm{dec}} W_Q, \qquad K = H^{\mathrm{enc}} W_K, \qquad V = H^{\mathrm{enc}} W_V.$$

Attention is then computed as:

$$\mathrm{CrossAttention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

No causal mask is applied here — the decoder is free to attend to any encoder position. Each decoder token queries the full source representation and retrieves what is most relevant for predicting the next target token.

Decoder Layer Structure. Each encoder–decoder decoder layer now contains three sub-blocks:

$$\begin{aligned} X' &= X + \mathrm{MaskedMHA}(\mathrm{LayerNorm}(X)), \\ X'' &= X' + \mathrm{CrossMHA}(\mathrm{LayerNorm}(X'), H^{\mathrm{enc}}), \\ Y &= X'' + \mathrm{FFN}(\mathrm{LayerNorm}(X'')). \end{aligned}$$

The masked self-attention maintains autoregressive generation over the target sequence, cross-attention retrieves relevant source context, and the FFN refines the representation locally.

Summary. In sequence transduction, the encoder builds a full contextual representation of the source without masking, while the decoder generates the target autoregressively, attending to both its own past outputs and the encoder representation through cross-attention. The layer-by-layer refinement described earlier applies to both the encoder and the decoder — the Transformer transforms both sequences into progressively richer representations until prediction becomes easy.

BERT 2018 , Developed by Google in 2018, it is a revolutionary machine learning framework for natural language processing (NLP) that uses a transformer-based, encoder-only architecture to understand the context of words in search queries and text.

Definition 1.17: GPT (Generative Pre-trained Transformer)

GPT is a decoder-only Transformer that uses causal masked self-attention and is trained autoregressively with the objective

$$\max_{\theta} \sum_{t} \log p_\theta(x_t \mid x_1, \ldots, x_{t-1}).$$

Advantages over RNNs:

- **Parallel training:** all positions are processed at once, removing the sequential bottleneck during training.
- **Direct long-range access:** any two positions interact through a single attention step, avoiding repeated transformations across time.
- **No compression bottleneck:** instead of storing history in a fixed-size hidden state, attention retrieves relevant information directly.
- **Scalability:** performance improves reliably with more data, parameters, and compute.

Generation remains sequential: token t must be produced before token $t + 1$.

The key conceptual shift from RNNs to attention is a move from *compression* to *retrieval*. An RNN must summarize the entire past into a single hidden state before knowing what future steps will need. Attention postpones this decision: each step directly retrieves whatever past information is most relevant.

Historical Evolution of Ideas Behind the Transformer

The Transformer architecture did not emerge in isolation. It is the result of several lines of research that progressively shaped its components.

- **Recurrent Neural Networks (1990s)** Early sequence models such as RNNs introduced the idea of processing tokens sequentially while accumulating history in a hidden state. This established the need for contextual representations.
- **LSTM and GRU (1997–2014)** Gated recurrent models addressed long-range dependencies and enabled deep stacked sequence models. The practice of stacking multiple layers later reappears in Transformer architectures.

- **Encoder–Decoder Seq2Seq Models (2014)** Neural machine translation introduced the encoder–decoder paradigm, where an input sequence is encoded into representations used by a decoder to generate outputs. The original Transformer preserves this high-level structure.
- **Early Attention Mechanisms (2014–2015)** Attention replaced the fixed-size context vector with dynamically computed weighted combinations of encoder states. This removed the bottleneck of compressing all information into a single vector and directly inspired self-attention.
- **MADE and Autoregressive Masking (2015)** Masked feed-forward networks demonstrated that autoregressive generation can be enforced using structural masking. Transformer decoders adopt the same principle through causal masking in self-attention.
- **Residual Networks (2015)** Skip connections enabled stable training of deep networks. Transformers apply residual connections around both attention and feed-forward blocks.
- **Layer Normalization (2016)** Layer normalization stabilized deep sequence models and became a standard component of Transformer layers.
- **Feed-Forward Attention Models (2016)** Work on attention-based models without recurrence showed that sequence modeling could be parallelized, motivating fully attention-based architectures.
- **Convolutional Seq2Seq Models (2017)** Fully parallel sequence models using convolution demonstrated that recurrence was not strictly necessary, paving the way for the Transformer's non-recurrent design.

Summary. The Transformer can be viewed as a synthesis of earlier ideas: stacked sequence modeling from RNNs, dynamic context via attention, autoregressive masking from feed-forward autoregressive models, residual learning from deep vision networks, and fully parallel computation from convolutional and attention-based sequence models.

1.6.7 Summary: Variable-Length Models

From recurrence to attention. LSTMs and GRUs solved the vanishing gradient problem but introduced a new one: the entire history of a sequence must be compressed into a fixed-size hidden state. This bottleneck is most visible in encoder–decoder models for machine translation, where performance degrades on long sentences because one vector cannot carry everything the decoder needs. Attention was introduced to alleviate this bottleneck: the decoder looks back at all encoder hidden states and forms a weighted sum, placing more weight on positions relevant to the current step. The weights are learned and input-dependent, giving the model soft control over what to retrieve — the same principle that made the LSTM cell state effective.

This idea turns out to be more than a patch. Recurrence was never the goal; it was simply a mechanism for modeling dependencies between positions by passing information forward step by step. Attention provides a different solution: each position can directly access any other, modeling dependencies in a single step rather than propagating them across many. This also removes the sequential constraint inherent in recurrence, allowing all positions to interact in parallel and making training much faster on long sequences.

One issue remains: causal order. During generation, the prediction at position i must depend

only on earlier positions. Recurrence enforces this naturally, but attention requires it to be imposed explicitly through masking (similar to the autoregressive masking used in MADE), so that information cannot flow from future to past positions.

The Transformer follows directly from these ideas: replace recurrence with attention as the primary mechanism for modeling dependencies, enforce causal structure through masking, and build the model around parallel, weighted aggregation of information combined with nonlinear transformations.